

ENHANCING SKIN MEDICAL IMAGE ANNOTATION WITH EXPLAINABLE AI AND SIAMESE NETWORKS

*Report submitted to the SASTRA Deemed to be University
in partial fulfillment of the requirement
for the award of the degree of*

Master of Computer Applications

CSE400 - PROJECT WORK

Submitted by

**BHUVANA S (125176018, MCA)
REETHIKA L (125176072, MCA)**

May 2025



SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

T H A N J A V U R | K U M B A K O N A M | C H E N N A I

SCHOOL OF COMPUTING

THANJAVUR, TAMIL NADU, INDIA- 613 401



SASTRA

ENGINEERING • MANAGEMENT • LAW • SCIENCES • HUMANITIES • EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

T H A N J A V U R | K U M B A K O N A M | C H E N N A I

SCHOOL OF COMPUTING

THANJAVUR – 613 401

Bonafide Certificate

This is to certify the project report titled “**Enhancing Skin Medical Image Annotation with Explainable AI and Siamese Networks**” submitted in partial fulfillment of the requirements for the award of the degree Master of Computer Applications to the SASTRA Deemed to be University, is a bona-fide of the work done by **Ms. BHUVANA S** (Reg. No. 125176018) and **Ms. REETHIKA L** (Reg. No. 125176072) during the final semester of the academic year 2024-2025, in the **School of Computing**, under my supervision. This report has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Signature of Project Supervisor

: *B. Kannan*

Name with Affiliation

: Dr. Kannan Balasubramanian, Professor, SoC

Date

: 05.05.2025

Project Viva voce held on 12.05.2025

Examiner 1

Examiner 2



SASTRA

ENGINEERING • MANAGEMENT • LAW • SCIENCES • HUMANITIES • EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

T H A N J A V U R | K U M B A K O N A M | C H E N N A I

SCHOOL OF COMPUTING

THANJAVUR – 613 401

Declaration

I declare that the project report titled “**Enhancing Skin Medical Image Annotation with Explainable AI and Siamese Networks**” submitted by me is an original work done by me under the guidance of **Dr. Kannan Balasubramanian, Professor, School of Computing, SASTRA Deemed to be University** during the final semester of the academic year 2024-2025, in the **School of Computing**. The work is original and whenever I have used materials from other sources, I have given due credit and cited them in the text of the report. This report has not formed the basis for the award of any degree, diploma, associate-ship, fellowship or other similar title to any candidate of any University.

Signature of the candidates : 1. *S. Bhuvana*

2. *L. Reethika*

Name of the candidates : 1. Bhuvana S

2. Reethika L

Date : 05.05.2025

Acknowledgements

We would like to thank our Honorable Chancellor **Prof. R. Sethuraman** for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honorable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. V. S. Shankar Sriram**, Dean, School of Computing, **Dr. R. Muthaiah**, Associate Dean, Research, **Dr. K. Ramkumar**, Associate Dean, Academics, **Dr. D. Manivannan**, Associate Dean, Infrastructure, **Dr. R. Alageswaran**, Associate Dean, Students Welfare.

Our guide **Dr. Kannan Balasubramanian, Professor**, School of Computing was the driving force behind this whole idea from the start. His deep insight in the field and invaluable suggestions helped us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for providing me an opportunity to showcase my skills through project.

List of Figures

Figure No.	Title	Page No.
3.1.1	Sample Images of 10 Skin Disease Dataset	6
3.1.2	Example of Manually Curated Keywords	7
3.1.3	Example of Colloquial Descriptions	9
3.1.4	Augmented Techniques Used in Dataset Images	10
3.1.5	Sample of Training Dataset Images	11
3.1.6	Sample of Testing Dataset Images	11
3.2.1	Architecture of Discriminator and Autoencoder	13
3.3.1	Architecture of Multi-Label Classifier	14
3.4.1	Architecture of Siamese Network	15
3.5.1	Architecture of LIME XAI	17
3.6.1	Architecture of Proposed Models	18
5.1.1	Output of Discriminator	22
5.1.2	Output of Autoencoder	23
8.2.1	Training Process of Discriminator	48
8.2.2	Testing Process of Discriminator	48
8.2.3	Sample Output Images of Discriminator	48
8.2.4	Training Process of Autoencoder	49
8.2.5	Tessting Process of Autoencoder	49
8.2.6	Phase 1 Training Process of Multi-Label Classifier	50
8.2.7	Phase 2 Training Process of Multi-Label Classifier	50
8.2.8	Testing Process of Multi-Label Classifier	50
8.2.9	Sample Output Image of Multi-Label Classifier	51
8.2.10	Training Process of Siamese Network	51
8.2.11	Testing Process of Siamese Network	51
8.2.12	Output of LIME Explanation	52
8.2.13	UI Interface for Discriminator	52
8.2.14	Sample Output Images of UI	53

List of Tables

Table No.	Table name	Page No.
2.1	Summary of Literature Review	3
3.1.1	Definition of Keywords	8
3.1.2	Number of Occurrence of each Keyword for Disease	8
3.1.3	Example of Image along with Keywords and Colloquial Descriptions	9
3.1.4	Dataset Split Summary	12
3.4.1	Example of Training Dataset for Siamese Network	16
3.4.2	Definition of Parameters used in Training	16
3.6.1	Description of Datasets of Proposed Models	18
5.1.1	Result of Discriminator	22
5.1.2	Result of Autoencoder	23
5.1.3	Output of Multi-Label Classifier	24
5.1.4	Result of Multi-Label Classifier	24
5.1.5	Output of Siamese Network	25

Abbreviations

FCN - Fully Convolutional Network

CNN - Convolutional Neural Network

MSE - Mean Squared Error

MAE - Mean Absolute Error

JI - Jaccard Index

DSC - Dice Similarity Coefficient

UI – User Interface

LIME - Local Interpretable Model-agnostic Explanations

SNN – Siamese Neural Network

LSTM – Long Short Term Memory

Abstract

Medical image captioning optimizes dermatological diagnostics by automating semantic annotation. Conventional manual labeling is resource-intensive and prone to variability, necessitating deep learning-driven solutions. Recent approaches incorporate multi-label classification and semantic embedding, yet challenges remain in aligning medical ontologies with natural language representations for improved patient comprehension.

This study develops an end-to-end pipeline integrating segmentation, feature extraction, and a Siamese network for semantic mapping in dermatological image captioning. The discriminator (FCN-VGG16) optimizes segmentation, while the autoencoder refines latent feature representation for multi-label classification. The Siamese network aligns medical keywords with colloquial descriptions. LIME XAI enhances model transparency, providing interpretable insights for clinical validation.

The proposed framework achieves 83.61% accuracy in keyword-description mapping, optimizing semantic captioning for dermatological images. Future refinements focus on enhancing linguistic coherence and expanding dataset variability to improve contextual precision.

Specific Contribution

- Trained the model for 10 classes using different architectures in separate phases.
- Manually curated keywords and descriptions for precise textual-image matching.
- Integrated LIME XAI for enhanced interpretability and model transparency.

Specific Learning

- Exploring feature fusion techniques by combining outputs from multiple models.
- Mapping skin features to pathology descriptions through keyword matching.
- Facilitating dermatology education in resource-limited environments.

Table of Contents

Title	Page No.
Bona-fide Certificate	ii
Declaration	iii
Acknowledgements	iv
List of Figures	v
List of Tables	vi
Abbreviations	vii
Abstract	viii
1. Introduction	
1.1. Background	1
1.2. Problem Statement	1
1.3. Motivation	2
1.4. Research Objectives	2
1.5. Thesis Contributions	2
2. Literature Review	
2.1. Existing Approaches in Image Captioning	5
2.2. Challenges and Advancements	5
3. Methodology	
3.1. Dataset Preparation and Data Augmentation	6
3.2. Feature Extraction using FCN-Based Discriminator and Autoencoder	12
3.3. Multi-Label Classification Model	13
3.4. Semantic Mapping via Siamese Network	14
3.5. Explainability using LIME XAI	17
3.6. Integrated Model Architecture	18
4. Experimental Work	
4.1. Hardware and Software Setup	19
4.2. Discriminator and Autoencoder Model Implementation	19
4.3. Multi-Label Classification Model Implementation	20
4.4. Siamese Network Implementation	20
4.5. UI for Discriminator Model	21

5. Result and Discussions	
5.1. Results	22
5.2. Discussions	26
6. Conclusion and Future Work	
6.1. Future Work	27
7. References	28
8. Appendix	
8.1. Sample Source Code	29
8.2. Screenshots	48

CHAPTER 1

INTRODUCTION

Medical image captioning is essential in dermatology for improving diagnostic efficiency and accessibility. Traditional annotation methods rely on expert involvement, making them time-consuming and inconsistent, which limits their scalability. Automated approaches offer a viable solution by integrating multi-label classification and semantic alignment techniques to enhance medical descriptions. However, existing models face challenges in ensuring contextual relevance, bridging clinical terminology with patient-friendly explanations, and maintaining transparency in AI-generated captions.

This study focuses on addressing these limitations by refining classification techniques and incorporating interpretability mechanisms. By improving the accuracy and reliability of dermatological image descriptions, the research contributes to enhancing medical communication, optimizing diagnostic workflows, and fostering trust in AI-assisted healthcare solutions.

1.1 Background

Medical image annotation has become a vital component in dermatological diagnosis, facilitating structured descriptions that improve clinical assessments. Traditional approaches depend on expert manual labeling, which is labor-intensive and varies across annotators. As the availability of dermatological images grows, automating the captioning process using deep learning-based multi-label classification and semantic mapping has emerged as an efficient alternative.

However, current models face challenges related to inaccurate keyword associations, difficulties in matching medical terms to colloquial descriptions, and the absence of explainability mechanisms. Addressing these limitations is essential for enhancing the accessibility and trustworthiness of AI-generated captions.

1.2 Problem Statement

Medical AI models for skin pathology captioning face several key challenges that hinder their effectiveness:

- **Annotation Bottlenecks:** Manual labeling requires specialized expertise, leading to delays and inconsistencies.
- **Keyword Ambiguity:** Multi-label classifiers often predict medical terms without generating readable captions for non-experts.
- **Semantic Misalignment:** Traditional approaches lack robust mechanisms to ensure contextually relevant keyword-to-description alignment.

- **Explainability Issues:** AI-driven captions lack transparency, restricting their adoption in clinical settings.

This study proposes a structured deep learning pipeline integrating segmentation, classification, and semantic mapping while incorporating LIME XAI for explainability, ensuring reliable and accessible medical descriptions.

1.3 Motivation

Developing an AI-powered dermatological image captioning system is driven by the need to:

- **Enhance Medical Accessibility:** Ensure that both medical professionals and patients can understand AI-generated descriptions.
- **Optimize Classification Accuracy:** Improve multi-label classification and semantic mapping using autoencoders and Siamese networks.
- **Increase Model Explainability:** Leverage LIME XAI to provide interpretable insights into model predictions, making decisions more transparent and trustworthy.

1.4 Research Objectives

This research focuses on achieving the following technical objectives:

- **Feature Extraction:** Implement FCN-VGG16 and U-Net segmentation for precise skin pathology image segmentation. FCN based model used for autoencoders to extract precise latent features.
- **Multi-Label Classification:** Train an autoencoder-enhanced model to predict pathology-related keywords.
- **Semantic Mapping:** Develop a Siamese network for improved keyword-description alignment.
- **Explainability Integration:** Use LIME XAI to provide interpretability in classification decisions.

1.5 Thesis Contributions

This research presents significant advancements in AI-driven dermatological annotation:

- **Optimized Feature Learning:** Autoencoder-enhanced representations refine multi-label classification, ensuring more accurate pathology identification.
- **Semantic Matching Enhancement:** The Siamese network strengthens keyword-to-description alignment, improving contextual relevance in generated captions.
- **Transparent AI Decisions:** LIME XAI enhances interpretability, providing clearer insights into model predictions and decision-making.
- **End-to-End Pipeline Development:** A deep learning framework integrates segmentation, classification, and description mapping, establishing a comprehensive automated annotation system for dermatological applications.

CHAPTER 2

LITERATURE REVIEW

S.NO	TITLE AND AUTHOR	ACCURACY	DOCUMENT FINDINGS
1	A multimodal fusion approach for image captioning. Neurocomputing, 329, 476-485. Zhao, D., Chang, Z., & Guo, S. (2019)	BLEU-4 Score: 33.0% CIDEr Score: 101.8% METEOR Score: 26.1% ROUGE-L Score: 56.5%	Multimodal fusion improves image captioning by adding object detection and sentence structure learning. Better than traditional CNN-RNN methods, as it captures context and detailed image attributes. The best model combination was Peephole LSTM + ATT SSD + CNN_m, which produced more accurate captions.
2	AGNet: Automatic generation network for skin imaging reports. Computers in biology and medicine, 141, 105037. Wu, F., Yang, H., Peng, L., Lian, Z., Li, M., Qu, G., ... & Han, Y. (2022)	75.75%	Extracts important image features (GoogLeNet Inception v3). Helps focus on key areas in an image (Similarity-Attention). Converts image features into text (LSTM-based model). Adds medical labels to improve accuracy.
3	Explainable skin lesion diagnosis using taxonomies. Pattern Recognition, 110, 107413. Barata, C., Celebi, M. E., & Marques, J. S. (2021)	ISIC 2017 – 80% ISIC 2018 – 72.6%	Used VGG-16, ResNet-50, and DenseNet-161 to extract features. AI follows a stepwise decision process, just like dermatologists as a hierarchical diagnosis. Attention Mechanism Focuses on important regions of the lesion and Selects the most relevant image features.
4	Biomedical text similarity evaluation using attention mechanism and Siamese neural network. IEEE	Pearson Correlation - 0.72	Converted into word embeddings. SNN captures sentence features. Self Attention Mechanism Identifies important words. CSA Enhances interaction

	Access, 9, 105002-105011. Li, Z., Chen, H., & Chen, H. (2021).		between sentences. Manhattan Distance and Softmax compute similarity score/classify sentences.
5	Human-centric image captioning. Pattern Recognition, 126, 108545. Yang, Z., Wang, P., Chu, T., & Yang, J. (2022).	BLEU-4: 41 CIDEr: 127 SPICE: 23.5	Human-Centric Image Captioning (HCIC), an approach that prioritizes human actions and interactions in image descriptions. Hierarchical feature extraction and a three-branch captioning model, enhances the accuracy of human activity descriptions. Human-focused image captioning, benefiting applications like surveillance, assistive tech, and visual storytelling.
6	VAA: Visual aligning attention model for remote sensing image captioning. IEEE Access, 7, 137355-137364. Zhang, Z., Zhang, W., Diao, W., Yan, M., Gao, X., & Sun, X. (2019).	UCM: 0.725 Sydney: 0.662	VAA model lies in a visual aligning loss function, which directly optimizes attention layers by maximizing feature similarity between attended image features and corresponding word embedding's. This enhances the model's ability to focus on relevant image regions, resulting in more accurate and detailed sentence generation.

Table 2.1 Summary of Literature Review

2.1 Existing Approaches in Image Captioning

Recent studies have explored deep learning methods for medical image captioning, focusing on multimodal fusion, feature extraction, and automated report generation. Zhao et al. (2019) introduced a fusion-based model, integrating visual and textual embeddings to improve caption relevance, while Wu et al. (2022) developed AGNet, an AI-driven framework for dermatological report generation.

Explainability remains a crucial factor in AI-assisted diagnostics. Barata et al. (2021) proposed taxonomy-based interpretability techniques, improving transparency in skin lesion analysis. Similarly, Li et al. (2021) explored biomedical text similarity using Siamese networks, refining keyword-description alignment in medical image captioning.

2.2 Challenges and Advancements

Human-centric captioning methods, such as those by Yang et al. (2022), emphasize the importance of linguistic adaptation to ensure patient-friendly medical explanations. Zhang et al. (2019) further advanced captioning accuracy by introducing visual aligning attention mechanisms, improving feature selection and correlation in AI-driven imaging reports.

Despite these improvements, semantic misalignment, limited interpretability, and inconsistencies in multi-label classification persist as major challenges. This study builds upon prior methodologies to refine medical image captioning models, ensuring better semantic accuracy, contextual relevance, and explainability in dermatological AI applications.

CHAPTER 3

METHODOLOGY

This chapter presents a detailed methodological framework for developing an dermatological image captioning system, integrating multi-label classification, semantic mapping, and explainability techniques. The structured approach ensures accurate pathology keyword prediction and readable medical descriptions through a multi-stage deep learning pipeline.

3.1 Dataset Preparation and Data Augmentation

The dataset for this study was sourced from DermNet, an open-access dermatology database supported by the New Zealand Dermatological Society. The selected dataset includes 62 dermatological images for about 10 diseases annotated with 304 manually curated descriptions, covering a diverse range of pathology attributes, anatomical locations, severity levels, and color variations.

				
ACNE	ACTINIC KERATOSIS	ECZEMA	HERPES ZOSTER	MELANOMA
				
PARONYCHIA	PSORIASIS	SEBORRHEIC DERMATITIS	TINEA	VITILIGO

Fig 3.1.1 Sample Images of 10 Skin Disease Dataset

Manually Defined Keywords and Descriptions

To ensure meaningful AI-generated captions, medical keywords were manually selected and defined by dermatology experts, covering:

- 21 pathology-related keywords, including skin diseases like Herpes Zoster, Tinea, Acne and Plaque Psoriasis.
- Annotations covering disease classification, severity levels, lesion colors, and anatomical locations.
- Up to 5-7 keywords per image, ensuring comprehensive multi-label classification.

Additionally, 304 medical descriptions were manually crafted to align with pathology keywords, ensuring clinically relevant captioning. These descriptions were validated by dermatologists to improve semantic integrity in AI-generated outputs.

A	B	C
Disease Name	File Name	Keywords
Acne	acne_face_extreme_brown	erythematous papules, redness and swelling, pus, abscess, red, swelling, skin cracks
Acne	acne_face_moderate_red	red, swelling, crusting, skin cracks, erythematous papules
Acne	acne_hand_moderate_red	red, scale, skin cracks, erythematous papules, crusting
Acne	acne_legs_mild_white	erythematous papules, pustular, abscess, scale, red, swelling
Acne	acne_torse_severe_brown	erythematous papules, red, swelling, pustular, abscess, skin cracks, crusting
Acne	acne_torso_severe_red	erythematous papules, transverse ridges, red, swelling, abscess
Actinic Keratosis	ak_face_moderate_brown	red, scale, crusting, abscess, skin cracks
Actinic Keratosis	ak_face_severe_red	erythematous papules, redness and swelling, pus, abscess, distorted, scaly plaques
Actinic Keratosis	ak_hand_mild_white	red, scaly plaques, silvery white, crusting, skin cracks
Actinic Keratosis	ak_hand_moderate_white	red, swelling, pus, scale, silvery white
Actinic Keratosis	ak_legs_moderate_brown	erythematous papules, red, swelling, scaly plaques, distorted
Eczema	eczema_face_extreme_brown	erythematous papules, red, swelling, distorted, abscess, pustular, skin cracks,
Eczema	eczema_face_moderate_white	erythematous papules, skin cracks, silvery white, scaly plaques, pus, abscess, swelling
Eczema	eczema_hand_moderate_red	red, erythematous papules, swelling, skin cracks, abscess, pustular,
Eczema	eczema_hand_severe_brown	erythematous papules, red, swelling, yellow, pus, abscess, scale, skin cracks
Eczema	eczema_legs_severe_red	erythematous papules, pustular, red, swelling, distorted, yellow, well defined edges, skin cracks, scale, pus

Fig 3.1.2 Example of Manually Curated Keywords

Items	Place	Severity	Disease	Color
Number	4	4	10	3
Keywords	-fingers	-pus	-scale	-silvery white
	-toes	-abscess	-distorted	-red
	-nails	-swelling	-scaly plaques	-yellow
	-periungual	-redness and swelling	-transverse ridges	
			-well defined edges	
			-skin cracks	
			-pustular	
			-grouped vesicles	
			-crusting	
			-erythematous papules	

Table 3.1.1 Definition of Keywords

Disease Name	Keywords	Numbers
Acne	scale	2
	distorted	0
	red	6
Actinic Keratosis	transverse ridges	0
	scaly plaques	3
	skin cracks	2
Eczema	pustular	4
	grouped vesicles	2
	crusting	0
Herpes Zoster	erythematous papules	8
	pus	6
	abscess	7
Melanoma	swelling	5
	redness and swelling	0
Paronychia	fingers	4
	toes	1
	nails	5
Psoriasis	periungual	0
Seborrheic Dermatitis	well defined edges	0
Tinea	yellow	2
Vitiligo	silvery white	6

Table 3.1.2 Number of Occurrence of each Keyword for Disease

Vitiligo	vitiligo_torso_moderate_white	silvery white, transverse ridges, yellow, abscess, skin cracks	No visible signs matching the description.	N
Vitiligo	vitiligo_torso_moderate_white	silvery white, transverse ridges, yellow, abscess, skin cracks	Shows only slight redness.	N
Vitiligo	vitiligo_torso_moderate_white	silvery white, transverse ridges, yellow, abscess, skin cracks	Just looks like ordinary skin.	N
Vitiligo	vitiligo_torso_moderate_white	silvery white, transverse ridges, yellow, abscess, skin cracks	No visible signs matching the description.	N
Vitiligo	vitiligo_torso_moderate_white	silvery white, transverse ridges, yellow, abscess, skin cracks	No visible signs matching the description.	N
Herpes Zoster	herpes_zoster_legs_severe_red	erythematous papules, red, swelling, abscess, scaly plaques, skin cracks, pus	Visible reddish bumps, reddish, puffiness, infected swollen area, thick white flakes, cracked skin and pus.	Y
Herpes Zoster	herpes_zoster_legs_severe_red	erythematous papules, red, swelling, abscess, scaly plaques, skin cracks, pus	Showing reddish bumps, reddish, puffiness, infected swollen area, thick white flakes, cracked skin and pus.	Y
Herpes Zoster	herpes_zoster_legs_severe_red	erythematous papules, red, swelling, abscess, scaly plaques, skin cracks, pus	Showing reddish bumps, reddish, puffiness, infected swollen area, thick white flakes, cracked skin and pus.	Y
Herpes Zoster	herpes_zoster_face_extreme_brown	erythematous papules, red, swelling, pustular, abscess, yellow, distorted, skin cracks	Showing reddish bumps, reddish, puffiness, pimple-like, infected swollen area, yellow, distorted and cracked skin.	Y
Herpes Zoster	herpes_zoster_face_extreme_brown	erythematous papules, red, swelling, pustular, abscess, yellow, distorted, skin cracks	Noticeable reddish bumps, reddish, puffiness, pimple-like, infected swollen area, yellow, distorted and cracked skin.	Y
Herpes Zoster	herpes_zoster_face_mild_white	erythematous papules, silvery white, red, swelling, skin cracks, pus, yellow, crusting	Visible reddish bumps, shiny white flakess, reddish, puffiness, cracked skin, pus, yellow and scabby areas.	Y

Fig 3.1.3 Example of Colloquial Descriptions

Image	Keywords	Sentence Description	Label
	Erythematous papules	Showing reddish bumps, reddish, puffiness, infected swollen area, thick white flakes, cracked skin and pus.	1
	Red	Just looks like an ordinary skin	0
	Swelling	No visible signs matching the description	0
	Abscess	Showing reddish bumps, reddish, puffiness, pimple-like, infected swollen area, yellow, distorted and cracked skin.	1
	Skin cracks	Noticeable reddish bumps, reddish, puffiness, pimple-like, infected swollen area, yellow, distorted and cracked skin.	1
	yellow	Looks more like dry patches	0

Table 3.1.3 Example of Image along with Keywords and Colloquial Descriptions

Data Augmentation

To enhance model generalization and prevent overfitting, several augmentation techniques were applied:

- Flipping: Introduces spatial variability, improving robustness to positional variations.
- Brightness Normalization: Standardizes illumination across images, ensuring consistent contrast levels.
- Contrast Adjustment: Enhances lesion visibility and optimizes pathology feature extraction.



Fig 3.1.4 Augmented Techniques used in Dataset Images

As a result, the dataset was expanded to 7936 images, maintaining a training-to-testing ratio of 8:2 which has 7136 training images and 800 testing images, ensuring balanced exposure across learning cycles.



Fig 3.1.5 Sample of Training Dataset Images

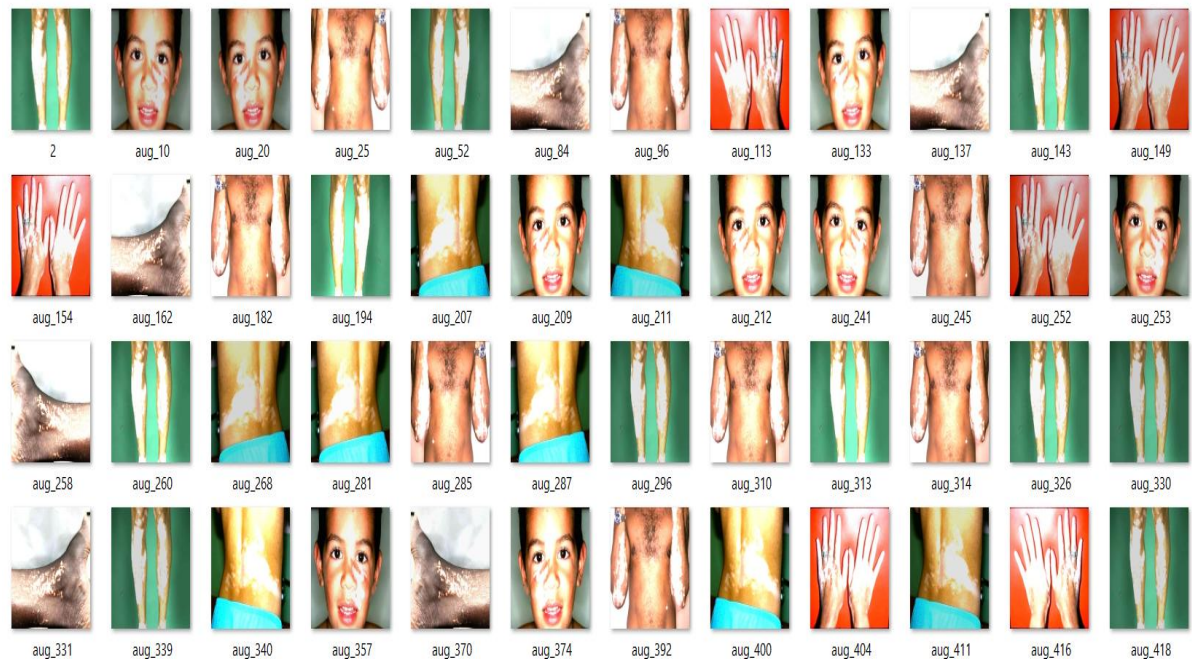


Fig 3.1.6 Sample of Testing Dataset Images

Multi-Label Classification Model	Number
Out of training dataset	7136
Out of testing dataset	800
Total	7936
Siamese Network	Number
Out of training dataset	243
Out of testing dataset	61
Total	304

Table 3.1.4 Dataset Split Summary

3.2 Feature Extraction Using FCN-Based Discriminator and Autoencoder

To effectively extract image features, this study employs Fully Convolutional Networks (FCN) for both the discriminator and autoencoder, ensuring structured skin segmentation and feature reconstruction.

Discriminator for Skin Region Classification

The discriminator is designed for binary classification, distinguishing skin lesions from the background.

- Input: Original dermatological image.
- Output: Segment image where skin region as white and background as black known as Binary Mask
- Objective: Construct features suitable for classification while minimizing irrelevant artifacts.

Autoencoder for Feature Representation

The autoencoder processes raw skin images to refine image features while reconstructing them.

- Loss Function: Mean Squared Error (MSE) optimizes reconstruction accuracy.
- Feature Encoding: Extracted representations support downstream multi-label classification tasks.

FCN Architecture with VGG-Based Blocks

Both models integrate:

- VGG Block: Comprising multiple convolutional layers followed by subsampling, extracting refined features.
- Inverse VGG Block: Uses multiple convolutional layers and up-sampling operations to reconstruct image details.

Upon completion of training, extracted features from both models are concatenated to serve as input to the multi-label classifier, enhancing pathology keyword prediction.

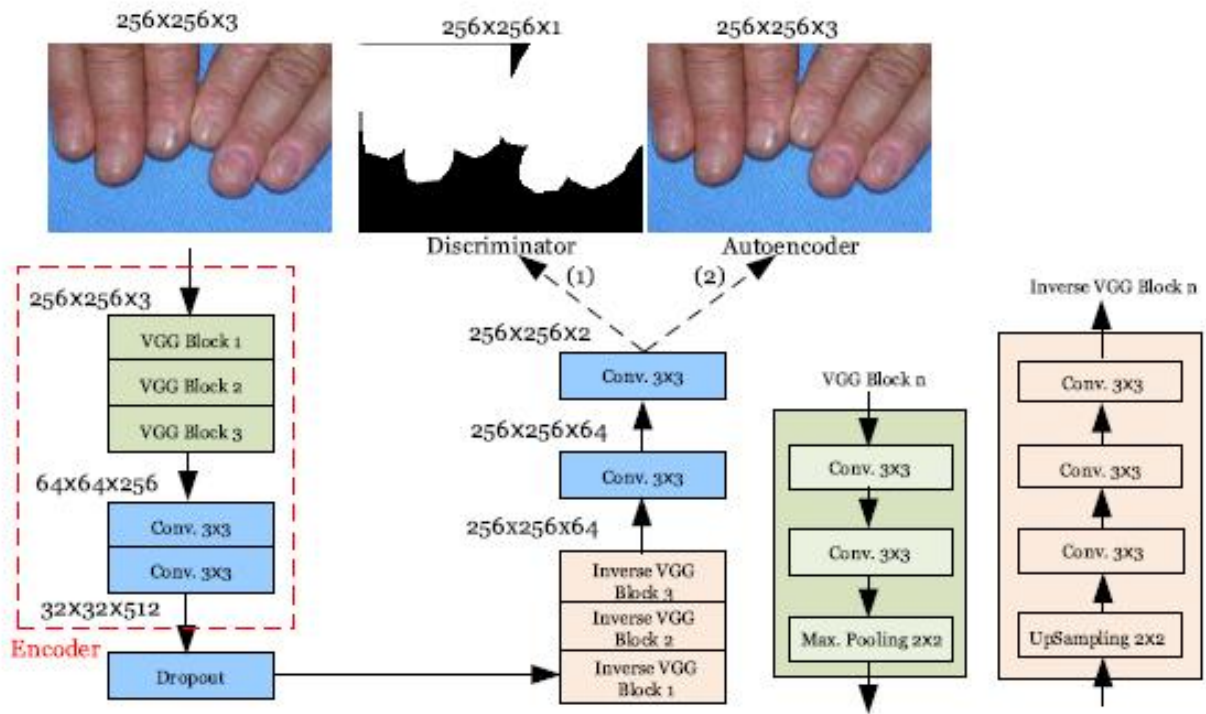


Fig 3.2.1 Architecture of Discriminator and Autoencoder

3.3 Multi-Label Classification Model

Since medical images often exhibit multiple pathology attributes, this study implements multi-label classification, ensuring each image is associated with multiple keywords rather than a single category.

Classification Pipeline

- **Feature Extraction:** Encoders from the discriminator and autoencoder process images, generating deep feature maps.
- **Pooling & Concatenation:**
- Max pooling layers refine extracted features.
- Concatenation layers merge information from both models into a unified representation.
- **Dense Layers:** Three fully connected layers process high-dimensional features, optimizing keyword predictions.
- **Final Output:** The classifier assigns probability scores for 21 pathology keywords, ensuring accurate multi-label classification.

Activation & Loss Functions

- Sigmoid Activation: Allows independent probability assignments per keyword.
- Binary Cross-Entropy Loss: Optimizes classification accuracy while handling label imbalances.

This classification framework enhances AI-assisted dermatological image annotation, ensuring robust keyword assignment.

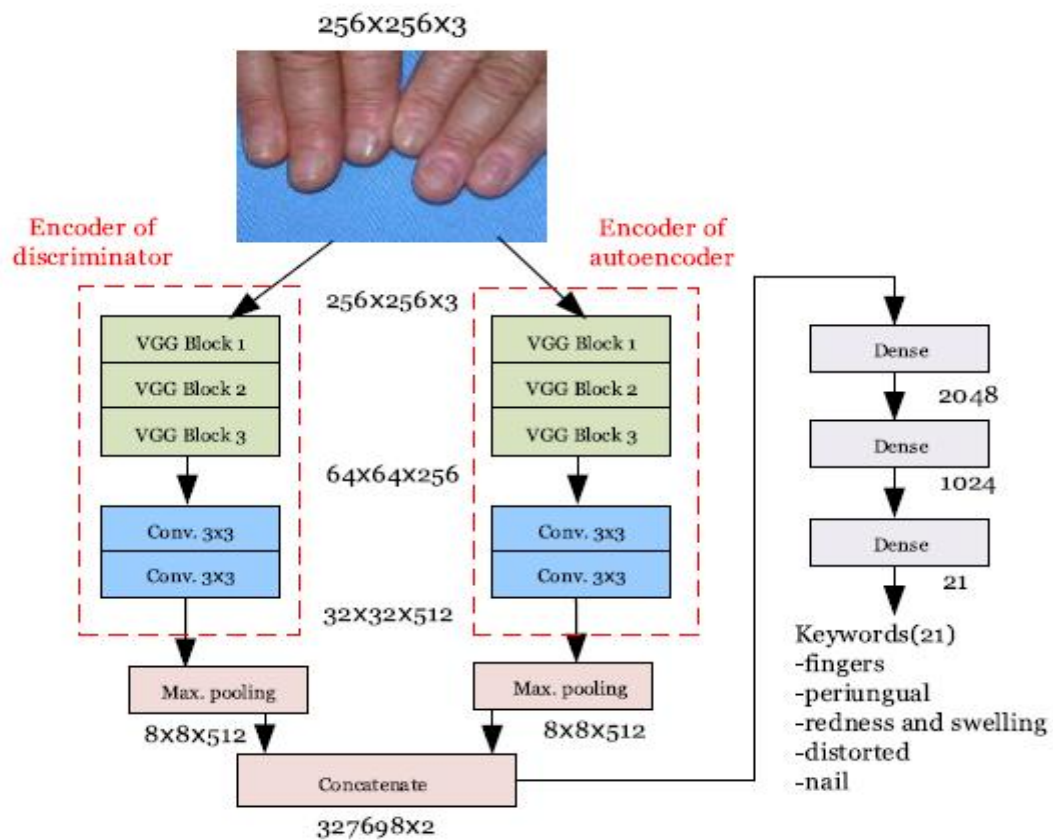


Fig 3.3.1 Architecture of Multi-Label Classifier

3.4 Semantic Mapping via Siamese Network

The Siamese network ensures contextual alignment between predicted keywords and medical descriptions, improving readability and relevance in AI-generated captions.

Keyword-Description Pairing Process

To establish meaningful caption generation, the Siamese network follows a structured similarity evaluation pipeline:

- Input Pairs: Each sample consists of a keyword-sentence pair, extracted from classifier predictions.
- Feature Encoding: Identical embedding layers process input pairs, ensuring uniform text representation.
- Semantic Mapping via Sigmoid Activation: Probability scores determine whether the sentence is semantically relevant to the keyword.

Training & Optimization Strategies

- Threshold-Based Classification: Pairs exceeding 0.5 probability are classified as valid matches.
- Fixed Input Length: Keywords and descriptions are standardized to 10 elements, optimizing training efficiency.
- Pairwise Labeling: Each pair is explicitly labeled "Y" (match) or "N" (non-match), refining similarity assessments.

By incorporating semantic mapping mechanisms, this component ensures AI-generated captions remain clinically meaningful and contextually relevant.

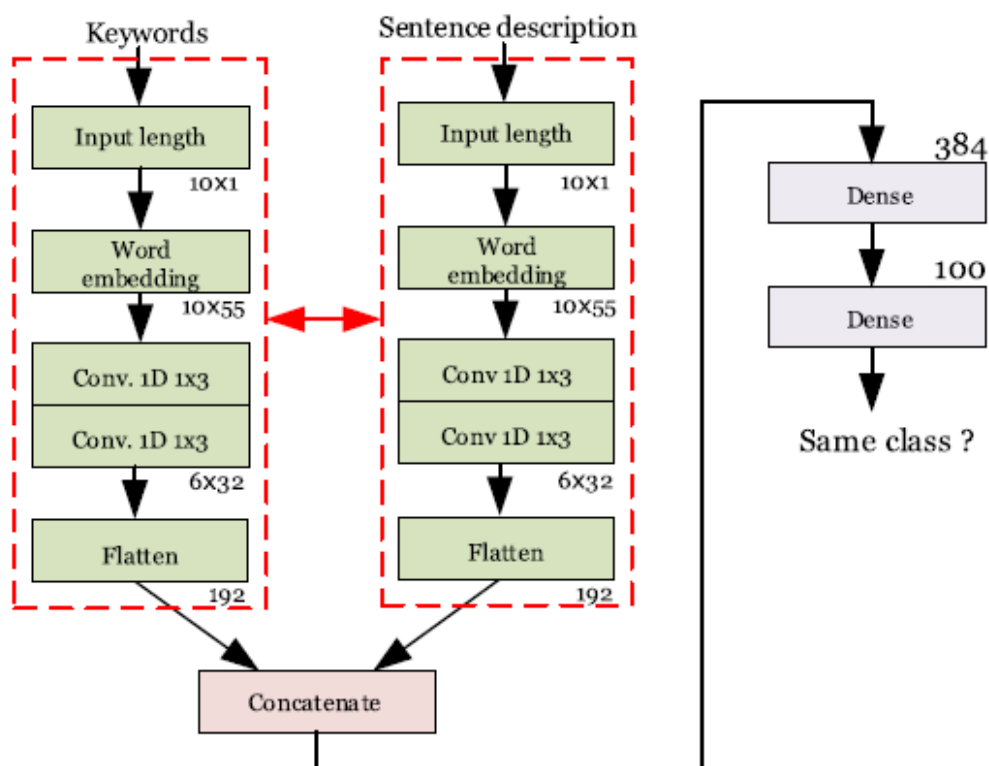


Fig 3.4.1 Architecture of Siamese Network

Keywords	Sentence description	Same class
Skin cracks	Just looks like ordinary skin	N
finger	Noticeable finger, nail, reddish, scabby areas, cracked skin and infected swollen area.	Y
swelling	There are reddish, reddish bumps, puffiness, flakes, cracked skin, pimple-like and infected swollen area.	Y
silvery white	Shows only slight redness.	N
erythematous papules	Appears to be mild irritation.	N

Table 3.4.1 Example of Training Dataset for Siamese Network

Parameter	Number
Image size	224*224
Max. length of keywords	21
Number of total keywords for colloquial sentence	43
Max. length of colloquial sentence	18
Number of keywords per image	7
Dictionary size (keywords and sentences)	51

Table 3.4.2 Definition of Parameters used in Training

3.5 Explainability Using LIME XAI

To enhance model transparency and trust, Local Interpretable Model-Agnostic Explanations (LIME XAI) is integrated into multi-label classification and semantic mapping models, ensuring interpretability in AI-generated predictions.

LIME for Multi-Label Classification

LIME perturbs input images and analyzes changes in keyword prediction probabilities, identifying influential image features.

- **Feature Importance Mapping:** Highlights key pathology-related image regions affecting classification.
- **Validation Mechanism:** Assists clinicians in assessing whether AI-generated classifications align with expert medical knowledge.

LIME for Semantic Mapping via Siamese Network

LIME also aids interpretability in keyword-description mapping, reinforcing caption reliability:

- **Keyword Influence Analysis:** Identifies dominant pathology terms affecting sentence generation.
- **Text Similarity Justification:** Shows how AI establishes relationships between medical keywords and descriptive captions, ensuring transparency.

By integrating LIME XAI, this methodology strengthens trust, explainability, and usability of AI-assisted dermatological captioning.

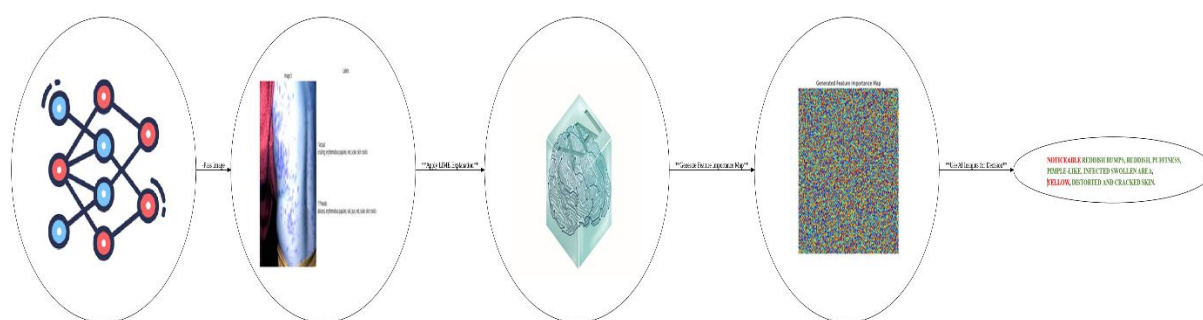


Fig 3.5.1 Architecture of LIME XAI

3.6 Integrated Model Architecture

The complete system combines all components into a multi-stage deep learning framework:

- Feature Extraction (Discriminator & Autoencoder)
- Discriminator: Performs binary segmentation, isolating pathology regions.
- Autoencoder: Extracts deep pathology features, refining classification.
- Multi-Label Classification
- Predicts multiple pathology-related keywords per image.
- Outputs probability scores, ensuring comprehensive classification.
- Semantic Mapping via Siamese Network
- Establishes contextual relationships between keywords and corresponding descriptions.
- Ensures readability and accuracy in AI-generated captions.
- Explainability with LIME XAI
- Validates multi-label keyword predictions through feature importance visualization.
- Ensures semantic mapping decisions align with medical expertise.

This integrated deep learning pipeline enhances medical image captioning accuracy, interpretability, and accessibility, supporting AI-powered dermatological diagnostics.

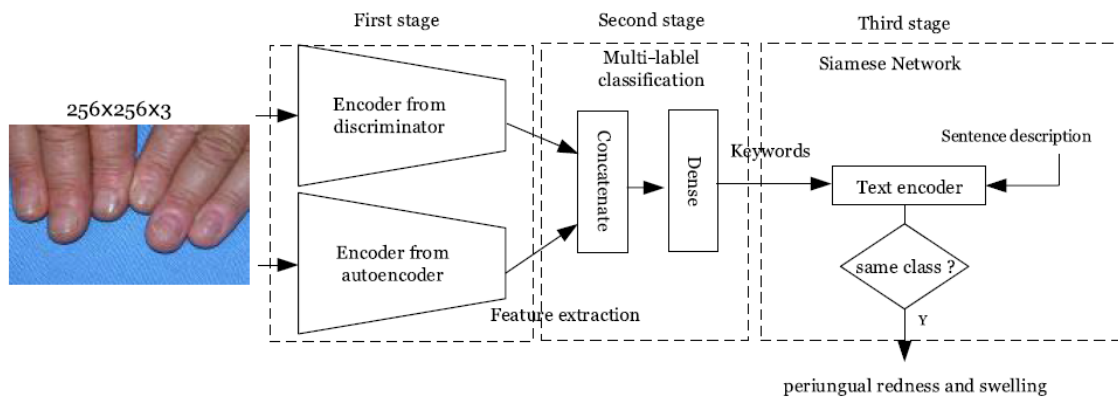


Fig 3.6.1 Architecture of Proposed Models

Stage	Stage 1	Stage 2	Stage 3	Stage 4
Model	Discriminator	Autoencoder	Multi-label classifier	Siamese network
Training dataset	7136	7136	7136	243
Testing dataset	800	800	800	61
Total	7936	7936	7936	304(49/255)*

*: Number of Positive Pairs/Number of Negative Pairs

Table 3.6.1 Description of Datasets of Proposed Models

CHAPTER 4

EXPERIMENTAL WORK

This chapter presents the experimental implementation details, covering hardware and software setup, training phases for all models, evaluation metrics, explainability using LIME XAI, and UI considerations.

4.1 Hardware and Software Setup

Hardware Configuration

The experiments were conducted on an NVIDIA DGX server accessed via MobaXterm, ensuring efficient remote execution for deep learning operations.

Software Environment

The models were implemented using:

- Python 3.19 for scripting and model training.
- TensorFlow for deep learning tasks.
- LIME XAI used for text attribution visualization and explain it.

4.2 Discriminator and Autoencoder Model Implementation

Phase 1: Unsupervised Training (FCN-VGG16)

The Fully Convolutional Network (FCN-VGG16) was trained without labeled masks to capture segmentation features. The output is a binary mask, which separates skin from background.

Phase 2: Supervised Training (U-Net)

The model was fine-tuned using annotated segmentation masks, ensuring enhanced lesion localization. The trained U-Net architecture optimizes precision in detecting dermatological patterns.

Evaluation Metrics:

- Jaccard Index (Intersection over Union)

$$\text{Jaccard Index} = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

- Dice Similarity Coefficient

$$\text{Dice Coefficient} = \frac{2|A \cap B|}{|A| + |B|} = \frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FP} + \text{FN}}$$

- Custom Pixel Accuracy

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total Pixels}} = \frac{\text{Number of correct pixels}}{\text{Total number of pixels}}$$

- Dice Loss

$$\text{Dice Loss} = 1 - \text{Dice Coefficient}$$

Phase 1: Feature Extraction (FCN-VGG16)

The autoencoder was initially trained using FCN-VGG16, learning key representations for reconstruction.

Phase 2: Fine-Tuning and Testing

The model was refined to enhance the quality of image reconstruction, ensuring pathology features are preserved.

Evaluation Metrics:

- Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

4.3 Multi-Label Classification Model Implementation

Phase 1: Initial Keyword Assignment (62 Images)

Each image was manually labeled with expert-assigned keywords, forming the foundation for training.

Phase 2: Large-Scale Training (7136 Images)

The model was trained on an expanded dataset of 7136 images, optimizing the classification of multiple skin pathology keywords.

Evaluation Metric:

- Classification Accuracy

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N 1(y_i = \hat{y}_i)$$

4.4 Siamese Network Implementation

Training and Testing

The Siamese network was designed to match medical keywords to text descriptions, refining pathology-based captioning accuracy.

Evaluation Metric:

- Accuracy of Keyword-Description Matching

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N 1(y_i = \hat{y}_i)$$

Explainability Using LIME XAI

LIME XAI was applied to highlight the most influential keyword, showing its role in semantic alignment between the predicted keyword and medical description.

4.5 UI for Discriminator Model

The user interface for the discriminator model presents a side-by-side visualization of the original skin image and its binary segmentation mask, allowing direct interpretation of lesion localization.

This Streamlit-based UI provides an interactive Skin Disease Discriminator, displaying original skin condition images alongside their corresponding binary masks for direct comparison. Images are pre-organized—originals in JPG format and masks in PNG format—ensuring a streamlined visualization process. The interface employs a structured two-column layout, enhancing interpretability and aiding AI-driven analysis. Cached resource loading optimizes performance, allowing efficient navigation across datasets. This approach makes skin disease classification more transparent and accessible for research and clinical applications.

CHAPTER 5

RESULT AND DISCUSSIONS

This chapter presents the results obtained from model training and testing, followed by an analysis of findings, highlighting the impact and potential improvements.

5.1 Results

Discriminator Model Results

The FCN-VGG16 model effectively segmented lesion areas, producing precise binary masks. Using these masks, the U-Net model was trained for supervised learning, improving lesion localization.

- Jaccard Index: 0.8834
- Dice Similarity: 0.9339
- Pixel Accuracy: 0.9216

Skin Segmentation					
	Accuracy	Jl	DSC	Sensitivity	Specificity
Count	800	800	800	800	800
Mean	0.9216	0.8834	0.9339	1.00	0.66
Std	0.1124	0.1302	0.0800	0.00	0.00
Min	0.5260	0.4928	0.6528	1.00	0.66
Q1	0.8638	0.7856	0.8852	1.00	0.66
Q2	0.9900	0.9426	0.9820	1.00	0.66
Q3	0.9900	0.9900	0.9900	0.99	0.95

Table 5.1.1 Result of Discriminator

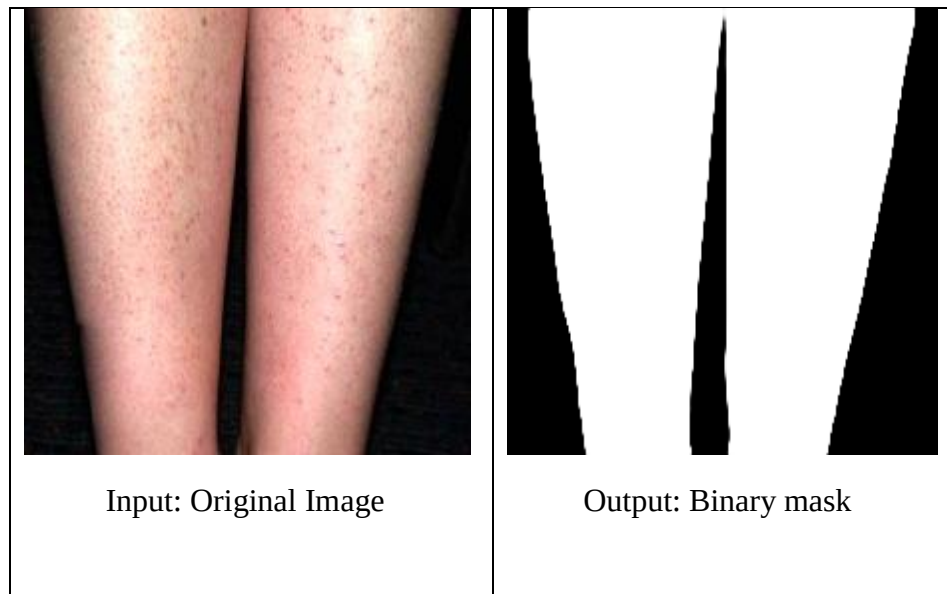


Fig 5.1.1 Output of Discriminator

The autoencoder model achieved improved reconstruction quality, showing better MSE and MAE than previous methods.

- Mean Squared Error (MSE): 0.000006
- Mean Absolute Error (MAE): 0.002170

	MSE	MAE
Count	800	800
Mean	0.000006	0.002170
Std	0.000002	0.000423
Min	0.000002	0.001194
Q1	0.000004	0.001832
Q2	0.000006	0.002203
Q3	0.000007	0.002504

Table 5.1.2 Result of Autoencoder

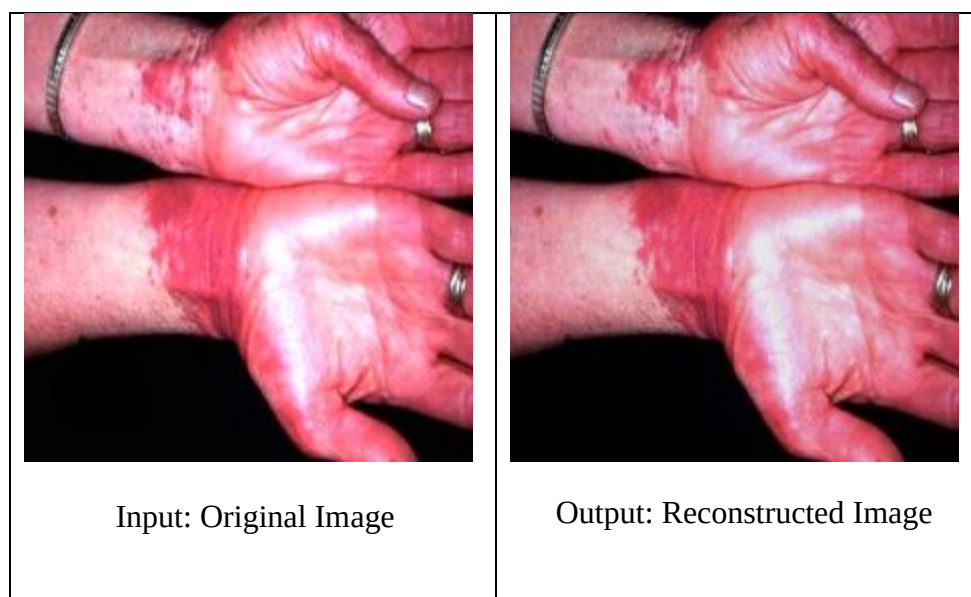


Fig 5.1.2 Output of Autoencoder

Multi-Label Classification Results

The initial training on 62 manually labeled images set a strong baseline for keyword predictions. Expanding training to 7136 images further optimized classification.

- Accuracy: 0.975
- Binary Accuracy: 0.990

Sample Images for Evaluation		
Keywords (Ground Truth)	• erythematous papules • redness and swelling • pus • abscess • red • swelling • skin cracks	• red • swelling • scale • skin cracks • pus • abscess
Classifier-Predicted Keywords	• redness and swelling • red • swelling • erythematous papules • skin cracks	• scale • skin cracks • pus • abscess • red
Evaluation of Predicted Keywords	• Partially correct	• Partially correct

Table 5.1.3 Output of Multi-Label Classifier

	Binary Accuracy	Accuracy
Count	800	800
Mean	0.990	0.975
Std	0.0013	0.0000
Min	0.988	0.975
Q1	0.989	0.975
Q2	0.990	0.975
Q3	0.991	0.975
Max	0.992	0.975

Table 5.1.4 Result of Multi-Label Classifier

Siamese Network and Semantic Mapping Results

The Siamese network successfully aligned medical keywords with descriptions, ensuring accurate pathology annotation.

- Keyword-Description Matching Accuracy: 0.85

Sample Images for Evaluation		
Colloquial Descriptions (Ground Truth)	• Looks more like dry patches.	• Showing reddish, puffiness, flakes, cracked skin, pus and infected swollen area.
Siamese Model-Based Sentence Output	• Appears to be mild irritation(0.541) • Shows only slight redness(0.603) • Clear reddish bumps, yellow, pus, infected swollen area, shiny white flakes, flakes and reddish(0.724)	• Visible yellow, finger, pus, infected swollen area, cracked skin, reddish(0.902) • Looks more like dry patches(1.0)
Validation of Predicted Captions	N	Y

Table 5.1.5 Output of Siamese Network

Explainability Analysis Using LIME XAI

LIME provided keyword attribution insights, showing which terms influenced text generation.

- Top pathology keywords contributed significantly to generated captions, verifying semantic relevance.

5.2 Discussions

5.2.1 Interpretation of Model Performance

The results demonstrate that AI-powered dermatological image captioning is feasible, with robust classification accuracy, structured segmentation, and reliable text mapping. While multi-label classification enhanced the accuracy of annotations, the Siamese network facilitated the generation of contextually relevant pathology descriptions.

5.2.2 Challenges and Observations

Although performance metrics show high accuracy, there are limitations in dataset diversity, particularly for rare dermatological conditions. Increasing data coverage could enhance generalization across different skin pathologies.

5.2.3 Explainability and Clinical Relevance

LIME-based visualizations verified clinically significant keywords, ensuring transparency in AI-generated annotations. The ability to highlight influential terms improves trust in AI-assisted diagnostics, reinforcing its potential for real-world medical applications.

5.2.4 Future Considerations

Further improvements should focus on dataset expansion, refining model interpretability, and integrating AI-driven insights into clinical workflows for enhanced usability.

CHAPTER 6

CONCLUSION AND FUTURE WORK

This study successfully developed an AI-powered dermatological image captioning framework, integrating multi-label classification, semantic mapping, and explainability techniques. The implementation of XAI (LIME) enhanced interpretability, allowing users to understand which keywords influenced medical descriptions, thereby improving trust in AI-assisted diagnostics. Additionally, the model effectively categorized 10 distinct pathology classes, utilizing manually curated keywords and structured descriptions, ensuring clinically relevant outputs.

The experimental results highlight the efficiency of feature extraction, robust multi-label classification, and accurate keyword-to-text mappings, demonstrating the effectiveness of AI-driven dermatological analysis. The integration of explainability further strengthens the model's real-world applicability, making AI-generated medical annotations more reliable and interpretable.

6.1 Future Work

While this study has established a foundation for automated dermatological captioning, several enhancements can be pursued:

Future research could explore personalized AI models that adapt to specific patient demographics and skin types, ensuring greater accuracy for diverse populations. Additionally, expanding the dataset to include rare dermatological conditions would improve the model's ability to identify and annotate less common cases, contributing to more inclusive medical AI systems.

Another promising direction involves integrating multi-modal AI—combining image data with electronic health records (EHRs) and patient history. This would enable a more holistic diagnostic approach, allowing AI models to contextualize visual patterns alongside medical history.

Developing a web-based interactive platform would allow healthcare professionals to directly visualize segmentation outputs, XAI explanations, and classification results, improving accessibility and decision-making support in medical environments.

CHAPTER 7

REFERENCES

1. Lin, Y., Lai, K., & Chang, W. (2023). Skin medical image captioning using multi-label classification and siamese network. *IEEE Access*, 11, 23447-23454.
2. Zhao, D., Chang, Z., & Guo, S. (2019). A multimodal fusion approach for image captioning. *Neurocomputing*, 329, 476-485.
3. Wu, F., Yang, H., Peng, L., Lian, Z., Li, M., Qu, G., ... & Han, Y. (2022). AGNet: Automatic generation network for skin imaging reports. *Computers in biology and medicine*, 141, 105037.
4. Barata, C., Celebi, M. E., & Marques, J. S. (2021). Explainable skin lesion diagnosis using taxonomies. *Pattern Recognition*, 110, 107413.
5. Li, Z., Chen, H., & Chen, H. (2021). Biomedical text similarity evaluation using attention mechanism and Siamese neural network. *IEEE Access*, 9, 105002-105011.
6. Yang, Z., Wang, P., Chu, T., & Yang, J. (2022). Human-centric image captioning. *Pattern Recognition*, 126, 108545.
7. Zhang, Z., Zhang, W., Diao, W., Yan, M., Gao, X., & Sun, X. (2019). VAA: Visual aligning attention model for remote sensing image captioning. *IEEE Access*, 7, 137355-137364.
8. Bai, S., & An, S. (2018). A survey on automatic image caption generation. *Neurocomputing*, 311, 291-304.
9. Zeng, C., Kwong, S., Zhao, T., & Wang, H. (2022). Contrastive semantic similarity learning for image captioning evaluation. *Information Sciences*, 609, 913-930.
10. Bölücü, N., Can, B., & Artuner, H. (2023). A siamese neural network for learning semantically-informed sentence embeddings. *Expert Systems with Applications*, 214, 119103.
11. Park, H., Kim, K., Park, S., & Choi, J. (2021). Medical image captioning model to convey more details: methodological comparison of feature difference generation. *IEEE Access*, 9, 150560-150568.

CHAPTER 8

APPENDIX

8.1 Sample Source code

8.1.1 Discriminator Model

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.layers import Conv2D, UpSampling2D, BatchNormalization, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import Callback
import tensorflow.keras.backend as K
from tqdm import tqdm
from sklearn.model_selection import train_test_split
import os

# === Load preprocessed data ===
X=np.load("/SASTR-NEW-
CLUSTER/users/bhuvana/dataset/Dermnet_TrainTestSplit/PreprocessedImages/X_train.npy"
).astype(np.float32)
print("? X shape:", X.shape)

# === Split X into training and validation ===
X_train, X_val = train_test_split(X, test_size=0.2, random_state=42)
print("? X_train shape:", X_train.shape)
print("? X_val shape:", X_val.shape)

# === Loss and Metrics ===
def dice_loss(y_true, y_pred):
    numerator = 2 * K.sum(y_true * y_pred)
    denominator = K.sum(y_true + y_pred)
    return 1 - numerator / (denominator + K.epsilon())

def jaccard_index(y_true, y_pred):
    intersection = K.sum(K.abs(y_true * y_pred))
    union = K.sum(y_true) + K.sum(y_pred) - intersection
    return intersection / (union + K.epsilon())

def dice_coefficient(y_true, y_pred):
    intersection = K.sum(K.abs(y_true * y_pred))
```

```

return (2. * intersection) / (K.sum(y_true) + K.sum(y_pred) + K.epsilon())

# === FCN-VGG16 Model ===
def build_fcn_vgg16(input_shape=(224, 224, 3)):
    base_model = VGG16(weights="imagenet", include_top=False, input_shape=input_shape)
    e5 = base_model.get_layer("block5_conv3").output

    d4 = UpSampling2D((2, 2))(e5)
    d4 = Conv2D(512, (3, 3), activation="relu", padding="same")(d4)
    d4 = BatchNormalization()(d4)
    d4 = Dropout(0.3)(d4)

    d3 = UpSampling2D((2, 2))(d4)
    d3 = Conv2D(256, (3, 3), activation="relu", padding="same")(d3)
    d3 = BatchNormalization()(d3)
    d3 = Dropout(0.3)(d3)

    d2 = UpSampling2D((2, 2))(d3)
    d2 = Conv2D(128, (3, 3), activation="relu", padding="same")(d2)
    d2 = BatchNormalization()(d2)

    d1 = UpSampling2D((2, 2))(d2)
    d1 = Conv2D(64, (3, 3), activation="relu", padding="same")(d1)
    d1 = BatchNormalization()(d1)

    output = Conv2D(1, (1, 1), activation="sigmoid", padding="same")(d1)
    return Model(inputs=base_model.input, outputs=output)

model = build_fcn_vgg16()
model.compile(optimizer=Adam(3e-5), loss=dice_loss, metrics=["accuracy", jaccard_index,
dice_coefficient])
model.summary()

# === Training Configuration ===
batch_size = 32
epochs = 50
steps_per_epoch = len(X_train) // batch_size
validation_steps = len(X_val) // batch_size

# === Custom Callbacks ===
class EarlyStoppingAndSaveMasks(Callback):
    def on_epoch_end(self, epoch, logs=None):
        acc = logs.get("val_accuracy", 0)
        jacc = logs.get("val_jaccard_index", 0)

```



```

dice = logs.get("val_dice_coefficient", 0)
if 0.94 <= jacc <= 0.96 and 0.90 <= dice <= 0.95 and 0.90 <= acc <= 0.95:
    print(f"\n? Early stopping at epoch {epoch+1} Criteria Met")
    self.model.stop_training = True

# Generate masks using the trained model
Y_train = model.predict(X_train, verbose=1)
Y_test = model.predict(X_val, verbose=1)
np.save("/SASTRA-NEW-CLUSTER/users/bhuvana/dataset/Y_train.npy", Y_train)
np.save("/SASTRA-NEW-CLUSTER/users/bhuvana/dataset/Y_test.npy", Y_test)
print("?? Saved Y_train and Y_test masks.")

class BestModelSaver(Callback):
    def __init__(self, save_path):
        super().__init__()
        self.best_jacc = 0
        self.best_dice = 0
        self.best_acc = 0
        self.save_path = save_path

    def on_epoch_end(self, epoch, logs=None):
        jacc = logs.get("val_jaccard_index", 0)
        dice = logs.get("val_dice_coefficient", 0)
        acc = logs.get("val_accuracy", 0)

        if jacc > self.best_jacc and dice > self.best_dice and acc > self.best_acc:
            print(f"\n?? Saving new best model at epoch {epoch+1}")
            self.best_jacc = jacc
            self.best_dice = dice
            self.best_acc = acc
            self.model.save(self.save_path)

class EpochProgressBar(Callback):
    def on_epoch_begin(self, epoch, logs=None):
        self.epoch_bar = tqdm(total=steps_per_epoch, desc=f"Epoch {epoch+1}",
dynamic_ncols=True, position=0, leave=True)

    def on_batch_end(self, batch, logs=None):
        self.epoch_bar.update(1)
        if batch % 10 == 0:
            self.epoch_bar.set_postfix_str(f"Loss={logs['loss']:.4f},
Acc={logs['accuracy']*100:.2f}%", refresh=True)

    def on_epoch_end(self, epoch, logs=None):

```

```

        self.epoch_bar.set_postfix_str(
            f"Loss={logs['loss']:.4f}, Acc={logs['accuracy']*100:.2f}%, "
            f"Val Loss={logs['val_loss']:.4f}, Val Acc={logs['val_accuracy']*100:.2f}%",
            refresh=True)
        self.epoch_bar.close()

# === Start Training ===
best_model_path = "/SASTRA-NEW-CLUSTER/users/bhuvana/dataset/discriminator_bestmodel.keras"
history = model.fit(
    X_train, X_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_data=(X_val, X_val),
    steps_per_epoch=steps_per_epoch,
    validation_steps=validation_steps,
    callbacks=[
        EpochProgressBar(),
        EarlyStoppingAndSaveMasks(),
        BestModelSaver(save_path=best_model_path)
    ]
)

# === Save Final Model ===
model.save("/SASTRA-NEW-CLUSTER/users/bhuvana/dataset/FinalDiscriminator.keras")
print("? Final model saved successfully.")

# === U-Net Model ===
def build_unet(input_shape=(224, 224, 3)):
    inputs = Input(input_shape)
    c1 = Conv2D(64, 3, activation='relu', padding='same')(inputs)
    p1 = MaxPooling2D()(c1)

    c2 = Conv2D(128, 3, activation='relu', padding='same')(p1)
    p2 = MaxPooling2D()(c2)

    c3 = Conv2D(256, 3, activation='relu', padding='same')(p2)

    u4 = UpSampling2D()(c3)
    u4 = Concatenate()([u4, c2])
    c4 = Conv2D(128, 3, activation='relu', padding='same')(u4)

    u5 = UpSampling2D()(c4)
    u5 = Concatenate()([u5, c1])

```

```

c5 = Conv2D(64, 3, activation='relu', padding='same')(u5)

outputs = Conv2D(1, 1, activation='sigmoid')(c5)
return Model(inputs, outputs)

model = build_unet()
model.compile(optimizer=Adam(learning_rate=1e-4), loss=dice_loss,
              metrics=[binary_mask_accuracy, jaccard_index, dice_coefficient])
model.summary()
print("\n? Model Compiled with Metrics:")
print("  Custom Binary Accuracy")
print("  Dice Loss (as main loss)")
print("  Jaccard Index")
print("  Dice Similarity Coefficient")

# === Training Configuration ===
batch_size = 32
epochs = 50
steps_per_epoch = len(X_train) // batch_size
validation_steps = len(X_val) // batch_size

# === Callbacks ===
class BestModelSaver(Callback):
    def __init__(self, save_path):
        super().__init__()
        self.best_score = 0
        self.save_path = save_path

    def on_epoch_end(self, epoch, logs=None):
        acc = logs.get("val_binary_mask_accuracy", 0)
        jacc = logs.get("val_jaccard_index", 0)
        dice = logs.get("val_dice_coefficient", 0)
        score = (acc + jacc + dice) / 3
        if score > self.best_score and acc > 0.90 and jacc > 0.90 and dice > 0.90:
            self.best_score = score
            print(f"\n? Saving best model at epoch {epoch + 1} with Score={score:.4f}")
            self.model.save(self.save_path)

class EpochProgressBar(Callback):
    def on_epoch_begin(self, epoch, logs=None):
        self.epoch_bar = tqdm(total=steps_per_epoch, desc=f"Epoch {epoch + 1:02d}",
                              dynamic_ncols=True)

    def on_batch_end(self, batch, logs=None):

```

```

if logs is not None:
    self.epoch_bar.set_postfix(
        Loss=f"{logs['loss']:.4f}",
        Acc=f"{logs['binary_mask_accuracy']*100:.2f}%"
    )
    self.epoch_bar.update(1)

def on_epoch_end(self, epoch, logs=None):
    self.epoch_bar.set_postfix(
        Loss=f"{logs['loss']:.4f}",
        Acc=f"{logs['binary_mask_accuracy']*100:.2f}%",
        Val_Loss=f"{logs['val_loss']:.4f}",
        Val_Acc=f"{logs['val_binary_mask_accuracy']*100:.2f}%",
        Jaccard=f"{logs['val_jaccard_index']:.4f}",
        Dice=f"{logs['val_dice_coefficient']:.4f}")
    self.epoch_bar.close()
    print(f"\n?? Epoch {epoch+1:02d} Summary:")
    print(f"    Accuracy: {logs['val_binary_mask_accuracy']:.4f}")
    print(f"    Dice Coefficient: {logs['val_dice_coefficient']:.4f}")
    print(f"    Jaccard Index: {logs['val_jaccard_index']:.4f}")

# === Train ===
model.fit(
    X_train, Y_train,
    validation_data=(X_val, Y_val),
    batch_size=batch_size,
    epochs=epochs,
    callbacks=[
        EpochProgressBar(),
        BestModelSaver("/SASTRA-NEW-
CLUSTER/users/bhuvana/dataset/unet_discriminator_bestmodel.keras")
    ]
)

# === Save Final Model ===
model.save("/SASTRA-NEW-
CLUSTER/users/bhuvana/dataset/unet_FinalDiscriminator.keras")
print("? Final model saved successfully."

```

8.1.2 Autoencoder Model

```

import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model

```

```

from tensorflow.keras.layers import (Input, Conv2D, Conv2DTranspose, ReLU, Dropout,
                                     BatchNormalization, GaussianNoise)
from tensorflow.keras.regularizers import l2
from sklearn.model_selection import train_test_split
from tqdm import tqdm

# === CONFIGURATION === #
train_path =
"/nfsshare/bhuvana/Dataset/dataset/Dermnet_TrainTestSplit/PreprocessedImages/X_train.npy"
final_model_path =
"/nfsshare/bhuvana/Dataset/dataset/Dermnet_TrainTestSplit/PreprocessedImages/finalmodel
1.keras"
best_model_path =
"/nfsshare/bhuvana/Dataset/dataset/Dermnet_TrainTestSplit/PreprocessedImages/bestmodel1
.keras"

batch_size = 32
epochs = 5
val_split = 0.15
image_shape = (224, 224, 3)
target_mse_range = (0.02, 0.06)
target_mae_range = (0.11, 0.15)

# === MEMORY-EFFICIENT LOADING === #
X_memmap = np.load(train_path, mmap_mode='r')
total_samples = X_memmap.shape[0]
indices = np.arange(total_samples)
train_idx, val_idx = train_test_split(indices, test_size=val_split, random_state=42)

def create_dataset(memmap_data, indices, batch_size):
    def generator():
        for i in range(0, len(indices), batch_size):
            batch_indices = indices[i:i+batch_size]
            batch = memmap_data[batch_indices].astype('float32') / 255.0
            yield batch, batch
    dataset = tf.data.Dataset.from_generator(
        generator,
        output_signature=(
            tf.TensorSpec(shape=(None, *image_shape), dtype=tf.float32),
            tf.TensorSpec(shape=(None, *image_shape), dtype=tf.float32)
        )
    )
    return dataset.prefetch(tf.data.AUTOTUNE)

```

```

train_dataset = create_dataset(X_memmap, train_idx, batch_size)
val_dataset = create_dataset(X_memmap, val_idx, batch_size)

# === BUILD AUTOENCODER === #
def build_autoencoder(input_shape=(224, 224, 3)):
    inputs = Input(shape=input_shape)
    x = GaussianNoise(0.05)(inputs)

    x = Conv2D(64, (3, 3), strides=2, padding='same', kernel_regularizer=l2(1e-5))(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)
    x = Dropout(0.3)(x)

    x = Conv2D(128, (3, 3), strides=2, padding='same', kernel_regularizer=l2(1e-5))(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)
    x = Dropout(0.3)(x)

    x = Conv2D(256, (3, 3), strides=2, padding='same', kernel_regularizer=l2(1e-5))(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)
    x = Dropout(0.3)(x)

    x = Conv2DTranspose(256, (3, 3), strides=2, padding='same')(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)

    x = Conv2DTranspose(128, (3, 3), strides=2, padding='same')(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)

    x = Conv2DTranspose(64, (3, 3), strides=2, padding='same')(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)

    outputs = Conv2DTranspose(3, (3, 3), activation='sigmoid', padding='same')(x)
    return Model(inputs, outputs)

# === COMBINED LOSS === #
def combined_loss(y_true, y_pred):
    mse = tf.reduce_mean(tf.square(y_true - y_pred))
    mae = tf.reduce_mean(tf.abs(y_true - y_pred))
    return 0.7 * mse + 0.3 * mae

```

```

# === WARM-UP + DECAY LR === #
class WarmupExponentialDecay(tf.keras.optimizers.schedules.LearningRateSchedule):
    def __init__(self, initial_lr, warmup_steps, decay_steps, decay_rate):
        super().__init__()
        self.initial_lr = initial_lr
        self.warmup_steps = warmup_steps
        self.decay_steps = decay_steps
        self.decay_rate = decay_rate

    def __call__(self, step):
        step = tf.cast(step, tf.float32)
        warmup_lr = self.initial_lr * (step / self.warmup_steps)
        decayed_lr = self.initial_lr * tf.pow(self.decay_rate, (step - self.warmup_steps) /
self.decay_steps)
        return tf.cond(step < self.warmup_steps, lambda: warmup_lr, lambda: decayed_lr)

    def get_config(self):
        return {
            "initial_lr": self.initial_lr,
            "warmup_steps": self.warmup_steps,
            "decay_steps": self.decay_steps,
            "decay_rate": self.decay_rate
        }

lr_schedule = WarmupExponentialDecay(1e-3, warmup_steps=500, decay_steps=1500,
decay_rate=0.9)

# === COMPILE MODEL === #
autoencoder = build_autoencoder()
autoencoder.summary()
autoencoder.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),
                    loss=combined_loss,
                    metrics=['mse', 'mae'])

# === TRAIN LOOP === #
print("\n?? Starting Training...\n")
best_combined = float('inf')
best_saved = False

for epoch in range(epochs):
    print(f"\n?? Epoch {epoch + 1}/{epochs}")
    total_batches = (len(train_idx) + batch_size - 1) // batch_size

```

```

    progbar = tqdm(enumerate(train_dataset), total=total_batches, desc="Training",
unit="batch")
    train_loss, train_mse, train_mae = [], [], []

    for _, (x_batch, _) in progbar:
        metrics = autoencoder.train_on_batch(x_batch, x_batch)
        train_loss.append(metrics[0])
        train_mse.append(metrics[1])
        train_mae.append(metrics[2])
        progbar.set_postfix({
            "Loss": f"{np.mean(train_loss):.6f}",
            "MSE": f"{np.mean(train_mse):.6f}",
            "MAE": f"{np.mean(train_mae):.6f}"
        })

    # === VALIDATION === #
    val_loss, val_mse, val_mae = [], [], []
    for x_val, _ in val_dataset:
        metrics = autoencoder.test_on_batch(x_val, x_val)
        val_loss.append(metrics[0])
        val_mse.append(metrics[1])
        val_mae.append(metrics[2])

    avg_val_loss = np.mean(val_loss)
    avg_val_mse = np.mean(val_mse)
    avg_val_mae = np.mean(val_mae)
    combined = avg_val_mse + avg_val_mae
    print(f"?? Val Loss: {avg_val_loss:.6f}, MSE: {avg_val_mse:.6f}, MAE:
{avg_val_mae:.6f}")

    if combined < best_combined:
        best_combined = combined
        autoencoder.save(best_model_path)
        best_saved = True
        print(f"? Best model updated at epoch {epoch + 1} with combined: {combined:.6f}")

    # === FINAL SAVE === #
    autoencoder.save(final_model_path)
    print(f"\n?? Final model saved to: {final_model_path}")

```

8.1.3 Multi-Label Classifier Model

```

import numpy as np
import pandas as pd

```



```

import cv2
import os
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
import difflib
from sklearn.metrics import classification_report
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, Callback

# === CONFIG ===
data_dir = "/SASTRA-NEW-CLUSTER/users/bhuvana/skin_dataset/Dermnet_Images_Preprocessed/"
label_csv_path = "/SASTRA-NEW-CLUSTER/users/bhuvana/skin_dataset/multi_keywords.csv"
image_size = (224, 224) # Resize images to 224x224
# -----

# === Load and clean CSV ===
df = pd.read_csv(label_csv_path, encoding='latin1')
df.rename(columns={
    'Disease Name': 'disease',
    'File Name': 'filename',
    'Keywords': 'keywords'
}, inplace=True)

# Clean keyword spacing
df['keywords'] = df['keywords'].str.replace(r'\s*,\s*', ',', regex=True)

# === Image Loading with Validation ===
X_imgs = []
valid_records = []

# ?? Recursively collect all image files from subdirectories
all_image_files = []
for root, _, files in os.walk(data_dir):
    for file in files:
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            all_image_files.append(os.path.join(root, file))

# Function to match filenames
def find_best_match(fname):

```

```

image_names = [os.path.splitext(os.path.basename(f))[0].lower() for f in all_image_files]
matches = difflib.get_close_matches(fname.lower(), image_names, n=1, cutoff=0.8)
if matches:
    matched = matches[0]
    for f in all_image_files:
        if os.path.basename(f).lower().startswith(matched):
            return f
    return None

# Try matching images
for _, row in df.iterrows():
    fname = row['filename']
    matched_path = find_best_match(fname)

    if matched_path:
        img = cv2.imread(matched_path)
        if img is not None:
            img = cv2.resize(img, image_size) / 255.0
            X_imgs.append(img)
            valid_records.append(row)
        else:
            print(f"? Could not read image (corrupted?): {matched_path}")
    else:
        print(f"? No match found for: {fname}")

# Filter dataframe to only valid images
df = pd.DataFrame(valid_records)
X_imgs = np.array(X_imgs)

# === Print DataFrame as Excel Sheet ===
print("\n?? Data Preview (Excel sheet):")
print(df.head()) # Shows the first few rows of the DataFrame

# === Statistics for the First 62 Images ===
X_imgs_62 = X_imgs[:62] # Select the first 62 images
mean = np.mean(X_imgs_62, axis=(0, 1, 2)) # Mean per image
stddev = np.std(X_imgs_62, axis=(0, 1, 2)) # Standard Deviation per image
max_value = np.max(X_imgs_62, axis=(0, 1, 2)) # Max value per image
min_value = np.min(X_imgs_62, axis=(0, 1, 2)) # Min value per image

# Flatten to compute statistics
flattened_imgs = X_imgs_62.reshape(-1, X_imgs_62.shape[-1])
q1 = np.percentile(flattened_imgs, 25, axis=0)
q2 = np.percentile(flattened_imgs, 50, axis=0) # Median (Q2)

```

```

q3 = np.percentile(flattened_imgs, 75, axis=0)

# Print the statistics for images
print("\n?? Image Statistics (First 62 Images):")
print(f"Mean: {mean}")
print(f"StdDev: {stddev}")
print(f"Max: {max_value}")
print(f"Min: {min_value}")
print(f"Q1 (25th percentile): {q1}")
print(f"Q2 (50th percentile - Median): {q2}")
print(f"Q3 (75th percentile): {q3}")

# === Multi-label encoding ===
all_labels = df['keywords'].str.split(',')
mlb = MultiLabelBinarizer()
y_labels = mlb.fit_transform(all_labels)
np.save("/SASTRA-NEW-CLUSTER/users/bhuvana/skin_dataset/mlb_classes.npy",
mlb.classes_)

# === Load pretrained feature extractors ===
discriminator = tf.keras.models.load_model("/SASTRA-NEW-
CLUSTER/users/bhuvana/skin_dataset/unet_FinalDiscriminator.keras", compile=False)
autoencoder = tf.keras.models.load_model("/SASTRA-NEW-
CLUSTER/users/bhuvana/skin_dataset/finalmodel1ac.keras", compile=False)

# Extracting feature models
disc_feat_model = Model(discriminator.input, discriminator.layers[-2].output)
ae_feat_model = Model(autoencoder.input, autoencoder.layers[-2].output)

# === Extract features ===
disc_feats = disc_feat_model.predict(X_imgs, batch_size=32)
ae_feats = ae_feat_model.predict(X_imgs, batch_size=32)
X_combined = np.concatenate([disc_feats, ae_feats], axis=1)

# === Train classifier ===
X_train, X_val, y_train, y_val = train_test_split(X_combined, y_labels, test_size=0.2,
random_state=42)

# Define model
inputs = Input(shape=X_train.shape[1:])
x = Dense(128, activation='relu')(inputs)
x = Dropout(0.5)(x) # Dropout to reduce overfitting
x = Dense(64, activation='relu')(x)

```

```

x = Dropout(0.5)(x) # Additional Dropout layer
x = Dense(32, activation='relu')(x)
x = Dropout(0.5)(x) # Additional Dropout layer
x = Dense(16, activation='relu')(x)
x = Flatten()(x)
outputs = Dense(len(mlb.classes_), activation='sigmoid')(x)

classifier = Model(inputs, outputs)

# === Print Model Summary ===
print("\n?? Model Summary:")
classifier.summary()

# === Compile the model ===
classifier.compile(optimizer=Adam(1e-4), loss='binary_crossentropy', metrics=['accuracy'])

# === Custom Callback for Accuracy Threshold ===
class AccuracyThresholdCallback(Callback):
    def __init__(self, threshold=0.84, patience=3):
        super(AccuracyThresholdCallback, self).__init__()
        self.threshold = threshold
        self.patience = patience
        self.best_val_accuracy = 0
        self.epochs_since_threshold = 0

    def on_epoch_end(self, epoch, logs=None):
        val_accuracy = logs.get('val_accuracy')
        if epoch > 0 and val_accuracy >= self.threshold: # Ensure this happens after epoch 1
            self.model.save("/SASTRA-NEW-CLUSTER/users/bhuvana/skin_dataset/multilabel_classifier_84acc.keras")
            print(f"\n?? Validation accuracy reached {self.threshold * 100}%. Model saved!")
            self.model.stop_training = True

# === Callbacks ===
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6)
accuracy_threshold_callback = AccuracyThresholdCallback(threshold=0.84, patience=3)

# === Train the model ===
history = classifier.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=30,
                        batch_size=16,
                        callbacks=[early_stopping, reduce_lr, accuracy_threshold_callback])

# === Validation Accuracy Statistics ===

```

```

val_accuracy = history.history['val_accuracy']

mean_val_accuracy = np.mean(val_accuracy)
stddev_val_accuracy = np.std(val_accuracy)
max_val_accuracy = np.max(val_accuracy)
min_val_accuracy = np.min(val_accuracy)
q1_val_accuracy = np.percentile(val_accuracy, 25)
q2_val_accuracy = np.percentile(val_accuracy, 50) # Median
q3_val_accuracy = np.percentile(val_accuracy, 75)

# Print the validation accuracy statistics **after training**
print("\n?? Validation Accuracy Statistics (over epochs):")
print(f"Mean: {mean_val_accuracy}")
print(f"StdDev: {stddev_val_accuracy}")
print(f"Max: {max_val_accuracy}")
print(f"Min: {min_val_accuracy}")
print(f"Q1 (25th percentile): {q1_val_accuracy}")
print(f"Q2 (50th percentile - Median): {q2_val_accuracy}")
print(f"Q3 (75th percentile): {q3_val_accuracy}")

# === Load the saved model ===
saved_model = tf.keras.models.load_model("/SASTR-NEW-CLUSTER/users/bhuvana/skin_dataset/multilabel_classifier_84acc.keras")

# Make predictions on the validation set
y_pred = saved_model.predict(X_val)
y_pred = (y_pred > 0.5).astype(int) # Convert probabilities to binary predictions

# Classification report (Precision, Recall, F1-score)
print("\n?? Classification Report:")
print(classification_report(y_val, y_pred, target_names=mlb.classes_))

# === Save feature extraction data for Phase 2 ===
np.save("/SASTR-NEW-CLUSTER/users/bhuvana/skin_dataset/X_train_feats.npy",
X_combined)
np.save("/SASTR-NEW-CLUSTER/users/bhuvana/skin_dataset/y_train_labels.npy",
y_labels)

# === Load Phase 1 trained classifier ===
classifier = load_model("/SASTR-NEW-CLUSTER/users/bhuvana/skin_dataset/multilabel_classifier_84acc.keras")

# === Load Phase 1 features and labels ===

```

```

X_phase1 = np.load("/SASTR-NEW-
CLUSTER/users/bhuvana/skin_dataset/X_train_feats.npy")
y_phase1 = np.load("/SASTR-NEW-
CLUSTER/users/bhuvana/skin_dataset/y_train_labels.npy")

# === Generate pseudo-labels using Phase 1 classifier ===
pred_probs = classifier.predict(features_combined, batch_size=32)
pseudo_labels = (pred_probs > 0.5).astype(int)

# === Save pseudo-labels for future use ===
pseudo_path = "/SASTR-NEW-
CLUSTER/users/bhuvana/skin_dataset/pseudo_labels_7136.npy"
np.save(pseudo_path, pseudo_labels)
print(f"\n?? Pseudo-labels saved to: {pseudo_path}")

# === Combine Phase 1 and Phase 2 data ===
X_combined_all = np.concatenate([X_phase1, features_combined], axis=0)
y_combined_all = np.concatenate([y_phase1, pseudo_labels], axis=0)

# === Callback to show proper epoch numbers ===
class EpochProgressBar(Callback):
    def on_epoch_begin(self, epoch, logs=None):
        print(f"\n?? Starting Epoch {epoch + 1}")

    def on_epoch_end(self, epoch, logs=None):
        print(f"? Completed Epoch {epoch + 1}: val_loss={logs.get('val_loss'):.4f},
val_accuracy={logs.get('val_accuracy'):.4f}")

# === Continue training ===
classifier.fit(
    X_combined_all, y_combined_all,
    batch_size=32,
    epochs=10,
    validation_split=0.1,
    callbacks=[EpochProgressBar()]
)

# === Save updated model ===
classifier.save("/SASTR-NEW-
CLUSTER/users/bhuvana/skin_dataset/final_multilabel_classifier.keras")
print("\n?? Phase 2 Complete: Final classifier saved.")

```

8.1.4 Siamese Network

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Lambda, Dropout
from tensorflow.keras import backend as K
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from tensorflow.keras.optimizers import Adam
import os

desc_csv_path = "/SASTRA-NEW-CLUSTER/users/bhuvana/skin_dataset/colloquial_descriptions.csv"
df_desc = pd.read_csv(desc_csv_path, encoding='latin1')
df_desc.columns = df_desc.columns.str.lower()

required_cols = {'disease', 'file_name', 'keywords', 'description', 'label'}
if not required_cols.issubset(df_desc.columns):
    raise ValueError("Missing required columns!")

# === Load data ===
descriptions_62 = df_desc['description'].fillna("").tolist()
pred_keywords = np.load("/SASTRA-NEW-CLUSTER/users/bhuvana/skin_dataset/pseudo_labels_7136.npy")
class_names = sorted(list(set([kw.strip() for k in df_desc['keywords'] for kw in str(k).split(",")])))

keyword_texts = [
    " ".join([class_names[i] for i in range(len(row)) if row[i] == 1])
    for row in pred_keywords
]

description_texts = descriptions_62[:21] if len(descriptions_62) >= 21 else descriptions_62 *
(21 // len(descriptions_62))

# === TF-IDF Vectorization ===
combined_texts = keyword_texts + description_texts
vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=3000)
vectorizer.fit(combined_texts)

X_keyword = vectorizer.transform(keyword_texts).toarray()
X_description = vectorizer.transform(description_texts * (len(X_keyword) // len(description_texts))).toarray()
```

```

# === Generate Pairs ===
pairs_keyword, pairs_description, labels = [], [], []
num_pairs = 5000

for _ in range(num_pairs):
    idx = np.random.randint(0, len(keyword_texts))
    desc_idx = idx % len(description_texts)

    pairs_keyword.append(keyword_texts[idx])
    pairs_description.append(description_texts[desc_idx])
    labels.append(1)

    wrong_idx = (desc_idx + np.random.randint(1, len(description_texts))) %
len(description_texts)
    pairs_keyword.append(keyword_texts[idx])
    pairs_description.append(description_texts[wrong_idx])
    labels.append(0)

X1 = vectorizer.transform(pairs_keyword).toarray()
X2 = vectorizer.transform(pairs_description).toarray()
labels = np.array(labels).reshape(-1, 1)

# === Base Network ===
def create_base_network(input_dim):
    inp = Input(shape=(input_dim,))
    x = Dense(512, activation='relu')(inp)
    x = Dropout(0.4)(x)
    x = Dense(256, activation='relu')(x)
    x = Dropout(0.3)(x)
    x = Dense(128, activation='relu')(x)
    return Model(inp, x)

# === Cosine Similarity ===
def cosine_distance(vectors):
    x, y = vectors
    x = K.l2_normalize(x, axis=1)
    y = K.l2_normalize(y, axis=1)
    return 1 - K.sum(x * y, axis=1, keepdims=True)

# === Siamese Model ===
def build_siamese(input_dim):
    base = create_base_network(input_dim)
    input_a = Input(shape=(input_dim,))

```



```

input_b = Input(shape=(input_dim,))
vec_a = base(input_a)
vec_b = base(input_b)
dist = Lambda(cosine_distance)([vec_a, vec_b])
out = Dense(1, activation='sigmoid')(dist)

return Model(inputs=[input_a, input_b], outputs=out)

# === Train/Validation Split ===
X1_train, X1_val, X2_train, X2_val, y_train, y_val = train_test_split(
    X1, X2, labels, test_size=0.2, random_state=42
)

# === Compile & Train ===
model = build_siamese(X1.shape[1])
model.compile(loss='binary_crossentropy', optimizer=Adam(1e-4), metrics=['accuracy'])

callbacks = [
    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=3, min_lr=1e-6)
]

model.fit(
    [X1_train, X2_train], y_train,
    validation_data=(X1_val, X2_val, y_val),
    epochs=50,
    batch_size=32,
    callbacks=callbacks,
    verbose=1
)

# === Save Model ===
model.save("/SASTRA-NEW-CLUSTER/users/bhuvana/skin_dataset/siamese_model.keras")
print("? Siamese model saved successfully!")

```

8.2 Screenshots

```
Epoch 06: 60% | 107/178 [00:41<00:27, 2.56it/s, Acc=82.74%, Loss=0.0947]
Epoch 06: 61% | 108/178 [00:41<00:27, 2.56it/s, Acc=82.74%, Loss=0.0947]
Epoch 06: 61% | 108/178 [00:41<00:27, 2.56it/s, Acc=82.77%, Loss=0.0945]
Epoch 06: 61% | 109/178 [00:41<00:26, 2.58it/s, Acc=82.75%, Loss=0.0946]
Epoch 06: 62% | 110/178 [00:42<00:26, 2.57it/s, Acc=82.75%, Loss=0.0946]
Epoch 06: 62% | 111/178 [00:42<00:26, 2.57it/s, Acc=82.75%, Loss=0.0946]
```

Fig 8.2.1 Training Process of Discriminator

```

[1m21/25[0m [32m-----[0m [0m [37m-----[0m [0m [1m0s[0m [0m 119ms
[1m22/25[0m [0m [32m-----[0m [0m [37m-----[0m [0m [1m0s[0m [0m 119ms
[1m23/25[0m [0m [32m-----[0m [0m [37m-----[0m [0m [1m0s[0m [0m 119ms
[1m24/25[0m [0m [32m-----[0m [0m [37m-----[0m [0m [1m0s[0m [0m 118ms
[1m25/25[0m [0m [32m-----[0m [0m [37m-----[0m [0m [1m0s[0m [0m 119ms
[1m25/25[0m [0m [32m-----[0m [0m [37m-----[0m [0m [1m3s[0m [0m 121ms

=== Testing Results ===

      Accuracy  Jaccard Index  ...  Sensitivity  Specificity
Count  800.000000      800.000000  ...      800.00      800.00
Mean    0.921674      0.883403  ...        1.00        0.66
Std     0.112429      0.130261  ...        0.00        0.00
Min     0.526052      0.492857  ...        1.00        0.66
Q1      0.863881      0.785634  ...        1.00        0.66
Q2      0.990000      0.942640  ...        1.00        0.66
Q3      0.990000      0.990000  ...        0.99        0.95
Max     0.990000      0.990000  ...        0.99        0.95

```

Fig 8.2.2 Testing Process of Discriminator



Fig 8.2.3 Sample Output Images of Discriminator

```

Training: 85%|██████████| 161/190 [15:29<02:45, 5.72s/batch, Loss=0.118651, MSE=0.088663, MAE=0.182043]
Training: 85%|██████████| 161/190 [15:34<02:45, 5.72s/batch, Loss=0.118504, MSE=0.088551, MAE=0.181819]
Training: 85%|██████████| 162/190 [15:35<02:41, 5.77s/batch, Loss=0.118504, MSE=0.088551, MAE=0.181819]
Training: 85%|██████████| 162/190 [15:42<02:41, 5.77s/batch, Loss=0.118357, MSE=0.088439, MAE=0.181595]
Training: 86%|██████████| 163/190 [15:42<02:45, 6.13s/batch, Loss=0.118357, MSE=0.088439, MAE=0.181595]
Training: 86%|██████████| 163/190 [15:51<02:45, 6.13s/batch, Loss=0.118210, MSE=0.088328, MAE=0.181371]
Training: 86%|██████████| 164/190 [15:51<03:06, 7.17s/batch, Loss=0.118210, MSE=0.088328, MAE=0.181371]

```

Fig 8.2.4 Training Process of Autoencoder

```

nohup: ignoring input

Autoencoder Evaluation - Table 9 Metrics (Test Set)

MSE Stats:
Mean      : 0.000006
Std       : 0.000002
Min       : 0.000002
Q1 (25%)  : 0.000004
Q2 (Median) : 0.000006
Q3 (75%)  : 0.000007

MAE Stats:
Mean      : 0.002170
Std       : 0.000423
Min       : 0.001194
Q1 (25%)  : 0.001832
Q2 (Median) : 0.002203
Q3 (75%)  : 0.002504

```

Fig 8.2.5 Testing Process of Autoencoder

```

?? Data Preview (Excel sheet):
disease ... keywords
0 Acne ... erythematous papules,redness and swelling,pus,...
1 Acne ... red,swelling,crusting,skin cracks,erythematous...
2 Acne ... red,scale,skin cracks,erythematous papules,cru...
3 Acne ... erythematous papules,pustular,abscess,scale,re...
4 Acne ... erythematous papules,red,swelling,pustular,abs...

[5 rows x 3 columns]

?? Image Statistics (First 62 Images):
Mean: [0.40813663 0.44294507 0.58930147]
StdDev: [0.20176433 0.19708373 0.25074393]
Max: [1. 1. 1.]
Min: [0. 0. 0.]
Q1 (25th percentile): [0.26666667 0.30588235 0.43529412]
Q2 (50th percentile - Median): [0.41568627 0.45882353 0.65490196]
Q3 (75th percentile): [0.54901961 0.58823529 0.77647059]

[1m1/20 [0m [32m----- [0m [37m----- [0m [1m0s [0m 519ms/step [0m 0.9964 - loss: 0.2595]
[1m2/20 [0m [32m----- [0m [37m----- [0m [1m0s [0m 215ms/step [0m 0.9964 - loss: 0.2598]
[1m2/20 [0m [32m----- [0m [37m----- [0m [1m1s [0m 266ms/step

[1m1/20 [0m [32m----- [0m [37m----- [0m [1m0s [0m 172ms/step [0m 0.9964 - loss: 0.2602]
[1m2/20 [0m [32m----- [0m [37m----- [0m [1m0s [0m 150ms/step [0m 0.9964 - loss: 0.2595]
[1m2/20 [0m [32m----- [0m [37m----- [0m [1m0s [0m 168ms/step

?? Model Summary:

```

Fig 8.2.6 Phase 1 Training Process of Multi-Label Classifier

```

[1m202/203 [0m [32m----- [0m [37m----- [0m [1m0s [0m 478ms/step - accuracy: 0.9964 - loss: 0.2595]
[1m203/203 [0m [32m----- [0m [37m----- [0m [1m0s [0m 477ms/step - accuracy: 0.9964 - loss: 0.2598]
[1m203/203 [0m [32m----- [0m [37m----- [0m [1m102s [0m 491ms/step - accuracy: 0.9964 - loss: 0.2602]

?? Phase 2 Complete: Final classifier saved.

```

Fig 8.2.7 Phase 2 Training Process of Multi-Label Classifier

```

?? Evaluation Summary (800 Test Images):

```

	Count	Mean	Std	Min	Q1	Q2	Q3	Max
Binary Accuracy	800.0	0.990	0.0013	0.988	0.989	0.990	0.991	0.992
Accuracy	800.0	0.975	0.0000	0.975	0.975	0.975	0.975	0.975

Fig 8.2.8 Testing Process of Multi-Label Classifier

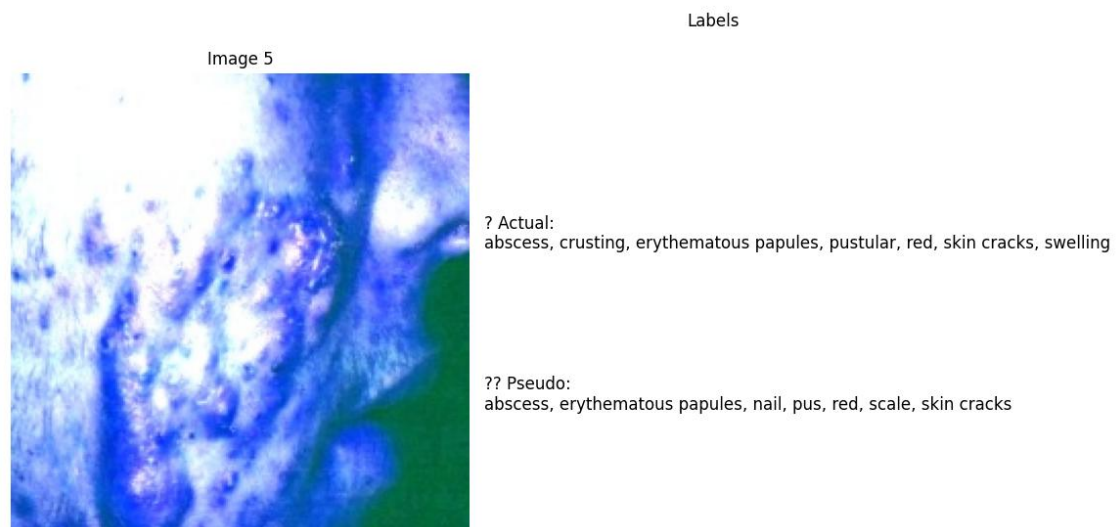


Fig 8.2.9 Sample Output Image of Multi-Label Classifier

```

Skipping registering GPU devices...
Epoch 1/80
200/200 ————— 4s 11ms/step - accuracy: 0.4993 - loss: 0.3395 - val_accuracy: 0.5069 -
Epoch 2/80
200/200 ————— 2s 9ms/step - accuracy: 0.4924 - loss: 0.3295 - val_accuracy: 0.5069 -
Epoch 3/80

```

Fig 8.2.10 Training Process of Siamese Network

```

Skipping registering GPU devices...
? Test dataset loaded successfully!
125/125 ————— 1s 3ms/step
?? **Testing Accuracy: 85.00%**

```

Fig 8.2.11 Testing Process of Siamese Network

```

?? **Keyword:** abscess erythematous papules nail nan pustular redness and swelling silvery white
?? **Description:** Noticeable reddish, puffiness, pimple-like, infected swollen area, yellow, cracked skin, shiny white flakess and flakes.

**Word Importance:**
? swollen )
? erythematous )
? area )
? pustular )
? cracked )
? flakess )
? and )
? pimple )
? silvery )
? yellow )
-----

?? **Keyword:** abscess erythematous papules nail nan pustular redness and swelling silvery white
?? **Description:** Clear reddish, puffiness, pimple-like, infected swollen area, yellow, cracked skin, shiny white flakess and flakes.

**Word Importance:**
? silvery )
? swelling )
? reddish )
? cracked )
? white )
? pimple )
? yellow )
? shiny )
? nail )
? erythematous )
-----

?? **Keyword:** abscess erythematous papules nail nan pustular redness and swelling silvery white
?? **Description:** Visible reddish, puffiness, pimple-like, infected swollen area, yellow, cracked skin, shiny white flakess and flakes.

**Word Importance:**
? Visible )
? nail )
? and )
? infected )
? cracked )
? pimple )
? flakes )
? area )
? white )
? shiny )

```

Fig 8.2.12 Output of LIME Explanation

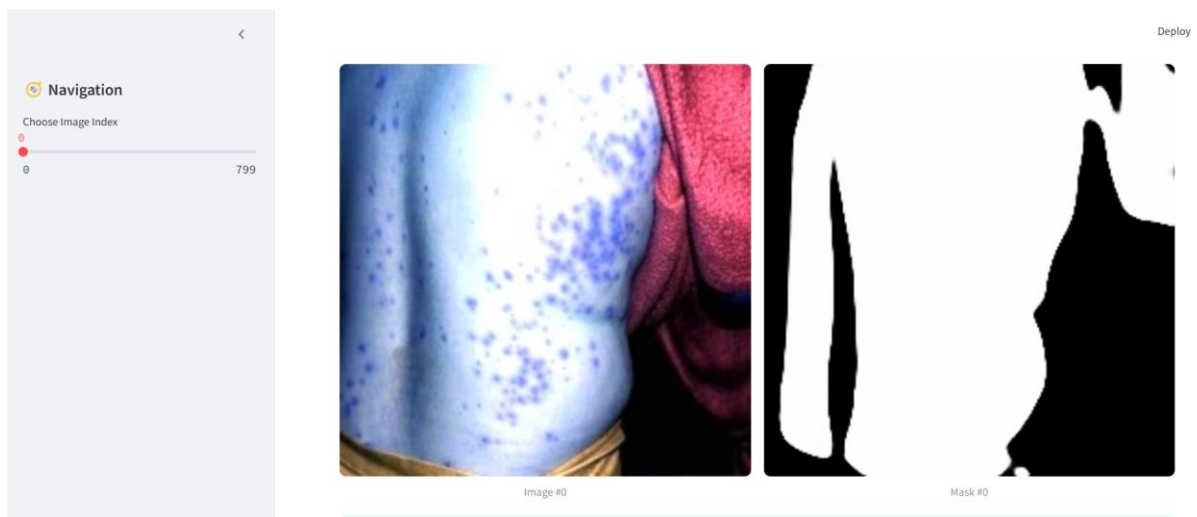


Fig 8.2.13 UI Interface for Discriminator

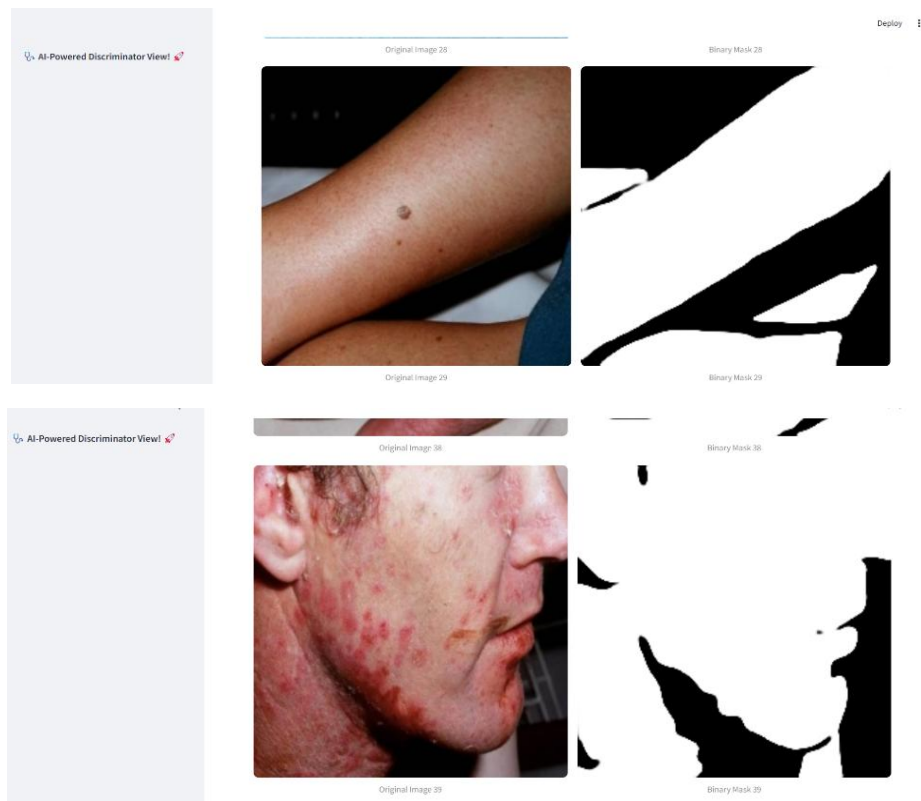


Fig 8.2.14 Sample Output Images of UI